國立臺灣大學電機資訊學院電機工程學研究所

碩士論文

Graduate Institute of Electrical Engineering

College of Electrical Engineering and Computer Science

National Taiwan University

Master’s Thesis

基於大型語言模型之暫存器傳輸級電路生成：代理式 IP 重用規劃與串接路由

Agentic IP-Reuse Planning and Cascade Routing for LLM-Based RTL Generation

（待填姓名）

TODO-YOUR-NAME

指導教授:（待填指導教授）博士

Advisor: TODO-ADVISOR-NAME, Ph.D.

中華民國 115 年 7 月

July 2026

doi:10.5566/NTU2026XXXXX

國立臺灣大學碩士學位論文

口試委員會審定書

基於大型語言模型之暫存器傳輸級電路生成：
代理式 IP 重用規劃與串接路由

Agentic IP-Reuse Planning and Cascade Routing
for LLM-Based RTL Generation

本論文係（待填姓名）君（R00000000）在國立臺灣大學電機工程學研究所完成之碩士學位論文，於民國 115 年 7 月 31 日承下列考試委員審查通過及口試及格，特此證明

口試委員：＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿

（指導教授）

＿＿＿＿＿＿＿＿＿＿　＿＿＿＿＿＿＿＿＿＿

＿＿＿＿＿＿＿＿＿＿　＿＿＿＿＿＿＿＿＿＿

＿＿＿＿＿＿＿＿＿＿　＿＿＿＿＿＿＿＿＿＿

＿＿＿＿＿＿＿＿＿＿　＿＿＿＿＿＿＿＿＿＿

所　　長：＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿＿

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Acknowledgements

（致謝內容待填。）

doi:10.5566/NTU2026XXXXX

摘要

大型語言模型（LLM）已能生成語法正確的程式碼，然而在真實硬體專案中，暫存器傳輸級（RTL）設計工作多以整合與矽智財（IP）重用為主，而非從零撰寫演算法模組。本論文以 RealBench 基準測試中源自三個真實開源 IP 專案（AES 加密核心、SD 卡控制器、E203 RISC-V 處理器核心）的 60 個模組級任務為評測對象，研究如何以系統工程手段——規劃、接地（grounding）、驗證、修復與路由——提升固定開源權重模型的 RTL 生成能力。

本論文提出兩項主要貢獻。其一為兩階段代理式 IP 重用流水線：規劃代理針對每個任務的真實 IP 目錄，以「建構式接地」（目錄注入、封閉詞彙表對齊、完整性檢查）產生可稽核的結構化重用計畫，再由生成器在忠實鏡射評分環境的 Verilator 驗證與修復迴圈下產生 Verilog。在溫度為零的對照實驗中，流水線通過率達 25.0%（語法 61.7%），為同一模型直接提示（10.0%／26.7%）的 2.5 倍，且增益完全集中於重用密集的 E203 家族。其二為兩層串接路由器，依任務性質在完整流水線與低成本直接生成之間逐任務選擇流程：在單變量消融中，解題數由 21/60 提升至 34/60，總運算時間反而由 213 小時降至 130 小時，每解一題的 token 效率為暴力混合策略的約 4 倍。

本論文同時以對照消融誠實量化各輔助機制：語意修復快取單調提升語法率（59.4%→66.9%）但完全不改變可解任務集合；兩種修復階段皆呈「速修或永不

doi:10.5566/NTU2026XXXXX

修」分布，深度為十的功能修復預算有 98% 的運算未產生任何通過；以黃金模型稽核法檢驗持續零分任務，發現並向上游回報一項測試平台競爭條件（已獲基準修復），亦確認其餘零分為真實失敗。整體結果顯示：當前瓶頸不在語法、規劃或模型規模，而在功能正確性——需要比失配計數更有效的回饋訊號方能突破。

關鍵字：大型語言模型、暫存器傳輸級設計、IP 重用、代理式規劃、模型路由、硬體描述語言

doi:10.5566/NTU2026XXXXX

Abstract

Large language models generate syntactically plausible code, but real-world RTL engineering is dominated by integration and IP reuse rather than freestanding algorithm writing. This thesis studies how far system engineering — planning, grounding, verification, repair, and routing around fixed open-weight models — can push LLM-based Verilog generation on RealBench, a benchmark of 60 module-level tasks drawn from three real open-source IP projects (an AES cipher core, an SD-card controller, and the E203 RISC-V processor core) and scored by a strict warnings-as-fatal compile-and-simulate harness.

The thesis makes two primary contributions. First, a two-stage agentic IP-reuse pipeline: a planning agent produces a structured, auditable reuse plan grounded by construction against a per-task catalog of the modules the environment actually provides — catalog injection, closed-vocabulary alignment, and a completeness gate, motivated by the finding that models never voluntarily invoke retrieval tools — followed by a genera-

doi:10.5566/NTU2026XXXXX

tor operating under a Verilator verify-and-repair loop that mirrors the scoring environment exactly. At temperature zero the pipeline passes 25.0% of tasks (61.7% syntax) against the same model's direct-prompt 10.0% (26.7%), a $2.5\times$ improvement concentrated entirely in the reuse-rich processor-core family. Second, a two-tier cascade router that matches task character to generation flow, sending integration-shaped tasks to the pipeline and self-contained algorithmic tasks to a cheap direct flow: in its controlled ablation it raises solved tasks from 21 to 34 of 60 while reducing total compute from 213 to 130 hours, reaching four times the token efficiency per solved task of brute-forcing both flows.

Controlled ablations quantify the supporting machinery with equal candor: a semantic repair cache lifts syntax monotonically (59.4%→66.9%) yet leaves task solvability exactly unchanged; both repair phases are fix-fast-or-never, with 98% of a depth-ten functional-repair budget producing no additional passes; and a golden-model audit of persistently failing tasks uncovered a testbench race condition — since fixed upstream by the benchmark — while confirming the remaining zeros as genuine. The binding constraint at the end is neither syntax, nor planning, nor model scale, but functional correctness, which will require a richer feedback signal than a mismatch count.

Keywords: Large Language Models, RTL Generation, IP Reuse, Agentic Planning, LLM Routing, Verilog

doi:10.5566/NTU2026XXXXX

Contents

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

List of Figures

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

List of Tables

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Denotation

LLM	Large Language Model（大型語言模型）
RTL	Register-Transfer Level（暫存器傳輸級）
HDL	Hardware Description Language（硬體描述語言）
IP	Intellectual Property core（矽智財；可重用之硬體設計區塊）
DUT	Device Under Test（受測設計；測試平台所例化之目標模組）
FSM	Finite-State Machine（有限狀態機）
CDC	Clock-Domain Crossing（跨時脈域）
RAG	Retrieval-Augmented Generation（檢索增強生成）
AES	Advanced Encryption Standard（進階加密標準）
SDC	SD-Card Controller（SD 卡控制器；本文之任務家族名稱）
E203	Nuclei HummingBirdv2 E203 開源 RISC-V 處理器核心
vLLM	高吞吐量 LLM 推論服務框架（PagedAttention）

doi:10.5566/NTU2026XXXXX

pass@k	k 次取樣中至少一次通過之無偏估計通過率
syntax rate	通過基準測試編譯（無任何錯誤與未豁免警告）之樣本比例
pass rate	同時通過編譯與測試平台功能比對之樣本比例
s/f	修復預算表記：每樣本至多 s 次語法修復與 f 次功能修復
MODDUP	Verilator 錯誤類別：重複宣告環境已提供之模組
PINNOTFOUND	Verilator 錯誤類別：例化埠不存在（違反介面契約）

doi:10.5566/NTU2026XXXXX

Chapter 1 Introduction

1.1 Motivation

Large language models write software with startling competence, and an industry has grown around turning that competence into engineering practice. Hardware description languages look, superficially, like the next domain over: Verilog is text, compiles, and has testbenches. But the economics of hardware make correctness qualitatively harsher — an RTL bug that reaches silicon is not patched, it is respun — and the daily reality of hardware engineering differs from the coding-benchmark ideal in a way that matters more than syntax: real RTL work is dominated by integration and reuse, not by writing fresh algorithms. A working chip is assembled from verified IP blocks; the engineer's craft lies in instantiating existing modules correctly, honoring interface contracts pin for pin, and wiring subsystems whose specifications run to hundreds of pages [10].

Most evaluations of LLM-generated Verilog sidestep exactly this reality: they pose small, self-contained modules with no environment to integrate against [14, 18]. When we instead evaluate on RealBench [9] — 60 module-level tasks carved from three real IP projects, scored by a strict compile-and-simulate harness — a capable open-weight model prompted directly compiles on only 26.7 % of tasks and passes 10.0 %. The gap between that number and usefulness is the subject of this thesis. Our premise is that the gap is less

doi:10.5566/NTU2026XXXXX

about model intelligence than about the system around the model: whether it is told what already exists, held to the environment's interface contracts, verified against the scorer's own toolchain, and spared work it predictably cannot convert.

1.2 Problem Statement

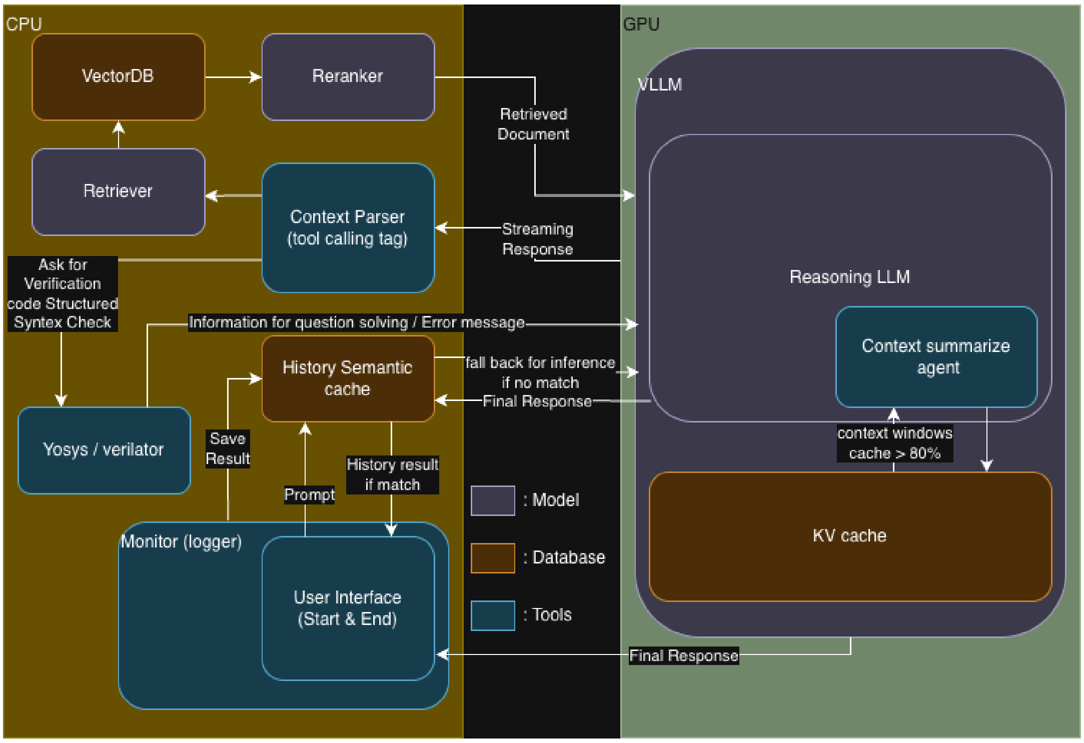

Figure 1.1: Deployment overview: CPU-side retrieval, planning, verification, and caching around a GPU-side local vLLM inference server.

Concretely, this thesis addresses the following problem. Given a natural-language specification of one Verilog module inside an existing project — with the project's other modules provided as source files, and a hidden-logic testbench fixing the module's exact port contract — generate RTL that compiles warning-free with the full project and passes the testbench, using only local open-weight models (Figure 1.1). Three properties make this hard. The environment imposes fatal structural contracts (re-declaring a provided module or missing a testbench port kills the sample outright). The tasks are heterogeneous

doi:10.5566/NTU2026XXXXX

— thin wrapper modules, algorithmic datapaths, and thousand-line integrators demand different generation strategies. And compute is finite and local: every planning step, repair attempt, and sample competes for the same GPU, so a system that cannot decide where its compute converts is a system that mostly wastes it.

1.3 Contributions

This thesis makes four contributions, each evaluated on the same 60-task benchmark under controlled ablations.

1. **A two-stage agentic IP-reuse pipeline** (Chapter 3): a planning agent that emits a structured reuse plan grounded — by construction, not by hoped-for tool use — against a per-task catalog of the modules the environment actually provides, followed by a generator operating under environment-faithful verification and repair. At temperature 0 the pipeline passes 25.0 % of tasks against the same model's direct 10.0 % ($2.5\times$; syntax $2.3\times$), with the entire gain concentrated in the reuse-rich processor-core family (Section 6.1).

2. **A cascade router between generation flows** (Chapter 4): a two-tier router — spec-level LLM pre-routing plus a plan probe for the uncertain residue — that sends integration-shaped tasks to the pipeline and self-contained algorithmic tasks to a cheap direct flow. In its controlled ablation it raises solved tasks from 21 to 34 of 60 while reducing total compute from 213 to 130 hours, and reaches $4\times$ the token efficiency per solved task of brute-forcing both flows (Section 6.7).

3. **Controlled measurements of the supporting machinery** (Chapter 6): a semantic

doi:10.5566/NTU2026XXXXX

repair cache that lifts syntax monotonically (59.4 %→66.9 %) yet leaves task solvability exactly unchanged; a repair-budget calibration showing both repair phases are fix-fast-or-never, with 98 % of a depth-ten functional budget producing nothing; and a functional repair loop quarantined as a separate arm because its feedback signal is the scoring oracle — an accounting line this literature rarely draws.

4. **A benchmark-artifact audit methodology and its findings** (Section 6.9): running golden implementations through the full scoring path to adjudicate persistent zeros. The audit uncovered a testbench race that scored provably byte-perfect S-box implementations as total failures — reproduced, root-caused to a coverage-instrumentation interaction, and subsequently fixed upstream in the benchmark — while confirming other zeros as genuine, and catching our own repair gate being stricter than the scorer on four tasks.

Throughout, negative and null results are reported with the same prominence as wins — the grounding fix that moved end-to-end quality by only $+0.6$ pp is, we argue, as informative as the router's $+13$ tasks.

1.4 Thesis Organization

Chapter 2 reviews the background and situates the thesis in four research threads. Chapters 3 and 4 present the pipeline and the router. Chapter 5 fixes the benchmark, models, metrics, and the inventory of every experimental run. Chapter 6 reports results: the headline comparison, the ablations, the routing study, a failure taxonomy, and the benchmark-artifact audit. Chapter 7 discusses implications, states threats to validity, and concludes. Appendices collect prompt templates and per-module result tables.

doi:10.5566/NTU2026XXXXX

Chapter 2 Background and Related Work

This chapter fixes the hardware-design background the thesis assumes (Section 2.1), then surveys the four research threads it builds on: LLMs for Verilog generation (Section 2.2), verification-in-the-loop repair (Section 2.3), retrieval-augmented code generation (Section 2.4), and LLM routing and cascades (Section 2.5). Section 2.6 positions this thesis against them.

2.1 RTL Design, Verification, and IP Reuse

Register-transfer-level (RTL) design expresses digital hardware as synchronous transfers between registers through combinational logic, written in a hardware description language — here Verilog/SystemVerilog [8]. Unlike software, RTL code carries a hard structural interface: a module's port list is a physical contract with its environment, and simulation semantics (blocking vs. nonblocking assignment, delta-cycle scheduling) can make two logically equivalent descriptions observably different to a testbench — a subtlety that returns with force in Section 6.9. Correctness is established by tools, not review: this thesis uses Verilator [27] for lint and compiled simulation and Yosys [31] for synthesis-based

doi:10.5566/NTU2026XXXXX

structural analysis.

Industrial hardware productivity rests on IP reuse: assembling verified, pre-existing blocks rather than rewriting them, a practice codified two decades ago [10]. A working chip team spends much of its effort not on new datapaths but on integration — instantiating existing modules and wiring their interfaces correctly. This observation is the design premise of Chapter 3: if real-world RTL work is reuse-dominated, an RTL-generating agent should plan reuse explicitly rather than synthesize every line from scratch. The open-source projects our benchmark draws from — an AES cipher core and an SD-card controller from OpenCores [23] and the HummingBirdv2 E203 RISC-V processor [19] — are themselves products of this reuse culture.

2.2 Large Language Models for Verilog Generation

Benchmarks. Evaluation infrastructure for LLM-generated Verilog began with fine-tuning-era studies [24, 28] and crystallized into standard suites: VerilogEval [14], machine-checked HDLBits-derived problems with the pass@k protocol inherited from HumanEval [5], later revised with specification-to-RTL tasks [25]; and RTLLM [18], larger open-ended designs judged on syntax, functionality, and quality. These suites consist predominantly of self-contained single modules. RTL-Repo [2] moved evaluation toward multi-file repository context, and RealBench [9] — used throughout this thesis (Section 5.1) — builds tasks from complete, real IP projects in which a target module must integrate against provided dependencies and survive a strict warnings-as-fatal harness. The distinction is not cosmetic: Chapter 6 shows that the phenomena that dominate RealBench (interface contracts, reuse, integration) are precisely the ones self-contained benchmarks cannot exhibit.

doi:10.5566/NTU2026XXXXX

Models and systems. One line of work improves the generator itself: VeriGen fine-tunes on harvested Verilog corpora [28]; ChipNeMo domain-adapts LLMs across chip-design tasks [13]; RTLCoder [16], CodeV [37], and CraftRTL [15] push open models past commercial baselines with synthetic-data pipelines, the last adding correct-by-construction non-textual representations and targeted repair data. Conversational studies such as Chip-Chat [3] probed interactive design with a human in the loop. This thesis is orthogonal to all of them: we hold the base models fixed (off-the-shelf open weights, Section 5.2) and engineer the system around the model — planning, grounding, verification, repair, and routing. Better base models slot in unchanged; indeed Section 6.8 measures exactly what a 6$\times$ model scale-up does and does not buy under a fixed system.

2.3 Verification-in-the-Loop Generation and Repair

Feeding tool feedback back into generation is the field's standard answer to the unreliability of one-shot code. In software, self-repair with execution feedback is well studied and its limits documented [20]; agentic framings add explicit reasoning-action loops [35], verbal self-reflection [26], and purpose-built agent-computer interfaces [34]. In hardware, AutoChip iterates generation against compiler and simulation errors [29]; RTL-Fixer targets syntax repair with retrieval over an error-pattern database [30]; MEIC frames RTL debug as an iterated LLM workflow [32]; and VerilogCoder couples an autonomous agent with graph-based planning and an AST/waveform-tracing tool, reporting strong VerilogEval-Human results [7].

The repair machinery of Section 3.4 belongs to this family but contributes three things the prior systems do not measure. First, a semantic repair cache that replays the model's

doi:10.5566/NTU2026XXXXX

own verified diagnostic→diff fixes across samples — related in spirit to RTLFixer's error-pattern retrieval, but populated online from verified successes rather than curated offline, and evaluated for what it changes (sample density) and what it cannot (task solvability, Section 6.4). Second, an explicit oracle-feedback accounting: our functional repair loop consumes the scoring testbench's mismatch signal, so it is quarantined as a separate arm — a methodological line most repair papers leave undrawn. Third, a budget calibration showing both repair phases are fix-fast-or-never (Section 6.6), turning repair-loop depth from folklore into a measured engineering parameter.

2.4 Retrieval-Augmented Code Generation

Retrieval-augmented generation grounds model output in external sources [12]; for code, retrieval over repositories supplies cross-file context that no prompt window holds natively, whether by similar-code lookup [17] or iterative repository-level retrieval interleaved with generation [36]. Our per-task IP catalog (Section 3.1) is a deliberately narrow instance: retrieval is scoped to the task's own dependency closure — never across tasks, eliminating benchmark contamination by construction — and its product is not free-form context but a closed vocabulary of instantiable modules that the planner's output is grounded against. The empirical twist (Section 6.2) is that the retrieval tool went unused by every model tested; what mattered was injecting the catalog and enforcing the vocabulary post-hoc — retrieval as constraint, not as context.

doi:10.5566/NTU2026XXXXX

2.5 LLM Routing and Cascades

A parallel literature allocates queries across models of different cost: FrugalGPT sequences models with a learned stopping rule [4]; Hybrid LLM trains a router to send easy queries to a small model [6]; RouteLLM learns routers from human preference data [21]; AutoMix self-verifies before escalating [1]. All of these route between models on general text tasks, assuming the larger model weakly dominates in quality. Our router (Chapter 4) differs in both axes. It routes between workflows over the same model — a cheap direct flow and an expensive agentic pipeline — and the expensive flow does not dominate: on self-contained algorithmic modules the pipeline is actively harmful (Section 4.1), so routing is about matching task character to flow shape, not buying quality with compute. Its cascade structure is also domain-specific: the mid-tier probe inspects a structured artifact the pipeline produces anyway (the plan), so the escalation step costs one planner call rather than a full duplicate generation.

2.6 Positioning

Table 2.1: This thesis against the closest related systems. "Reuse planning" = explicit grounded IP-reuse decisions; "workflow routing" = per-task choice among generation flows.

System	Repair loop	Retrieval	Reuse planning	Workflow routing	Repo-realistic eval
VeriGen [28]	–	–	–	–	–
AutoChip [29]	✓	–	–	–	–
RTLFixer [30]	✓	✓	–	–	–
VerilogCoder [7]	✓	✓	–	–	–
RepoCoder [36]	–	✓	–	–	✓
RouteLLM [21]	–	–	–	model-level	–
This thesis	✓	✓	✓	✓	✓

To our knowledge, this thesis is the first system to combine catalog-grounded IP-

doi:10.5566/NTU2026XXXXX

reuse planning with per-task workflow routing for RTL generation, and the first to evaluate the combination on a repository-realistic benchmark under controlled ablations (Table 2.1). Equally deliberately, it contributes negative and diagnostic results the surveyed literature rarely reports: a grounding fix that works perfectly yet moves end-to-end quality by less than a point, a repair cache that changes density but not solvability, a functional-repair budget that is 98 % wasted at depth ten, and a benchmark-artifact audit that exonerated the models on two tasks and convicted our own tooling on four more (Section 6.9).

doi:10.5566/NTU2026XXXXX

Chapter 3 A Two-Stage Agentic IP-Reuse Pipeline

This chapter presents the first contribution of the thesis: a two-stage agentic pipeline that generates RTL by first planning which existing IP blocks to reuse and how to integrate them, and only then generating Verilog under a verify-and-repair loop. Section 3.1 gives the dataflow and the environmental contracts the pipeline must honor. Sections 3.2 and 3.3 describe the two stages. Section 3.4 describes the three repair mechanisms layered on the generator — the syntax repair loop with a semantic repair cache, the functional repair loop, and diagnostic-driven specification slicing. Design decisions are motivated here; their empirical evaluation is deferred entirely to Chapter 6.

3.1 Overview and Environmental Contracts

The pipeline processes one benchmark task end-to-end as follows (Figure 3.1):

1. **Catalog construction.** The task's own dependency sources — and only those; no cross-task material ever enters, so there is no benchmark contamination — are indexed into a per-task reusable-IP catalog: one entry per provided module, carrying its name, port list, and source.

doi:10.5566/NTU2026XXXXX

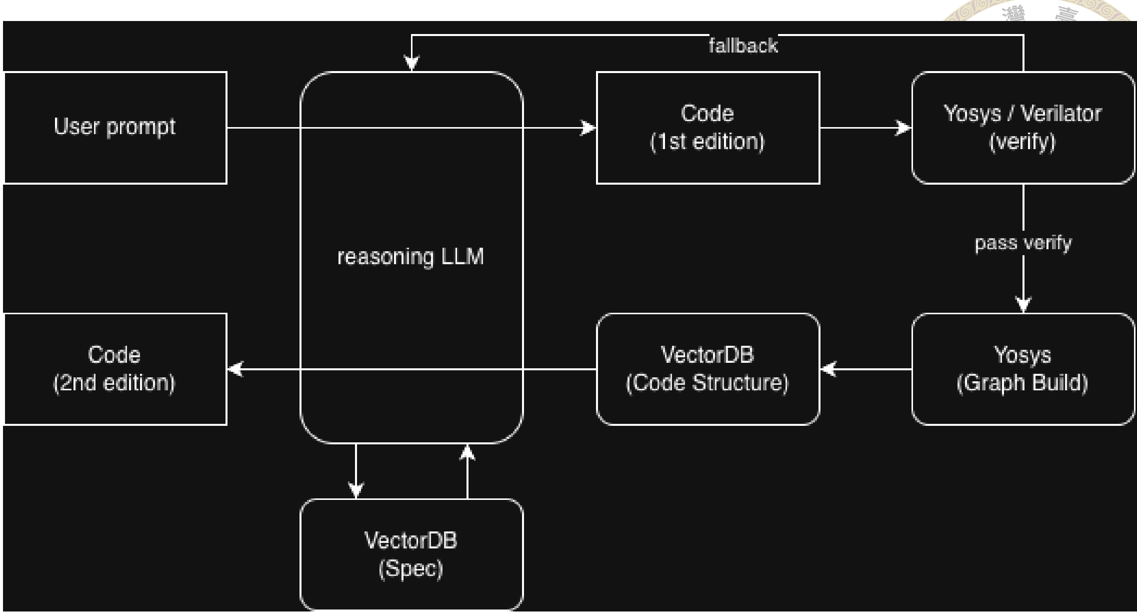

Figure 3.1: Dataflow of the two-stage pipeline. A RealBench task is expanded into a per-task IP catalog; the planner emits a grounded reuse plan; the plan adapter enriches it with source code and interface contracts; the generator produces Verilog inside a Verilator verify-and-repair loop before the final benchmark evaluation.

2. **Planning.** The planner (Section 3.2) reads the specification and the catalog and emits a structured IP-reuse plan: requirements, module decomposition, a reuse decision per submodule (reuse a catalog entry vs. write new RTL), and integration notes.

3. **Plan adaptation.** An adapter normalizes the plan, resolves each reuse decision to concrete source code, and assembles the generation prompt: the original specification, the signatures of reused modules, environment notes, and the testbench's DUT instantiation verbatim as the binding interface contract.

4. **Generation and repair.** The generator (Section 3.3) produces the target module and iterates under an internal Verilator lint that compiles the generated code against the same file set the benchmark will use, feeding diagnostics back as repair prompts (Section 3.4).

5. **Evaluation.** The final candidate is scored by the untouched RealBench Makefile

doi:10.5566/NTU2026XXXXX

as described in Section 5.1.

The two fatal contracts. Two properties of the evaluation environment shape nearly every design decision downstream, and both are fatal when violated. First, the environment compiles every Verilog file in the task directory together, so any provided dependency module that the generated code re-declares produces a duplicate-module error (`MODDUP`): shipped dependencies must be instantiated, never re-implemented. (The single family-wide exception, `e203_extend_csr`, is not shipped as a file and must be inlined — the kind of irregularity that motivates grounding plans in the real catalog rather than in the model's memory.) Second, the testbench instantiates the device under test with an exact port list; a generated module that misses one port fails with `PINNOTFOUND` regardless of how correct its internals are. The pipeline addresses the first contract by stripping re-declared modules and cataloging what is provided, and the second by injecting the testbench's DUT instantiation into every generation and repair prompt as a hard interface contract.

3.2 Stage One: The Grounded Planner

The planner is a tool-calling agent equipped with a `search_reuse_ip` tool over the per-task catalog (token-overlap, semantic-embedding, or hybrid retrieval backends). Given the specification, it must return a structured JSON plan with a fixed schema: requirement summary, module decomposition, per-module reuse decisions (`selected_ip` naming a catalog entry, or `new_rtl_required`), and integration, verification, and debugging plans.

doi:10.5566/NTU2026XXXXX

The models do not call tools. The single most consequential empirical fact about this stage is negative: across every configuration we ran, the planner never invoked its retrieval tool — 0 tool calls in 1,200 plans with gpt-oss-20B, and still 0 in 600 plans with the 6× larger gpt-oss-120B. The models simply emit the structured plan directly. Left uncorrected, the reuse decisions in those plans are hallucinated from pretraining memory: plausible-but-wrong module names such as `e203_exu_core` where the real catalog has `e203_exu`. Rather than fighting the refusal (prompt pressure, forced tool choice, and re-prompting all failed to change it), the final design removes the dependency on tool use with three mechanisms that together we call grounding-by-construction:

1. **Catalog injection.** The real catalog vocabulary — every reusable module id with a one-line digest — is injected directly into the planner's prompt, and the system prompt mandates a closed vocabulary: `selected_ip` must be one of the listed ids.

2. **Closed-vocabulary grounding.** Every `selected_ip` in the returned plan is post-hoc resolved against the catalog: exact matches kept; near misses remapped by prefix/substring matching and then fuzzy string similarity (cutoff 0.6); irrecoverable names dropped and the submodule marked `new_rtl_required`. The plan additionally records the grounding outcome per decision, making plans auditable.

3. **Completeness gate.** A task whose catalog is non-empty but whose plan contains no reuse and no integration content is re-prompted once (tool-free), and the more complete of the two attempts is kept.

Chapter 6 (Section 6.2) audits the effect: grounding eliminates hallucinated names from plans entirely, and — an equally important honest finding — barely moves the end-to-end pass rate, because the generator was already neutralizing bad names downstream.

doi:10.5566/NTU2026XXXXX

Plans become correct and auditable; the binding constraint lies elsewhere.

Large specifications. Specifications beyond a token threshold (45,000 estimated tokens) are condensed before planning by a chunk-and-merge summarizer that renders two views from one manifest: a planner view (interfaces, reuse queries, and a brief behavioral digest) and a generation view (full requirements for to-be-generated logic; interface-only for provided IP). Port tables and module headers are carried verbatim from the raw specification past the LLM summarizer, because interface fidelity is exactly what the two fatal contracts punish. Condensation compresses the largest specifications by roughly $5\times$ (e.g., $\sim$720 k $\rightarrow$ $\sim$146 k characters) while keeping every shared fact and module interface stated exactly once.

3.3 Stage Two: The Generator

The generator receives the adapted plan and produces the target module with a code-focused prompt assembled from: the (possibly condensed) specification; the source and signatures of every reused module; the environment notes (which modules are provided as files, which macros the environment defines); and the testbench DUT instantiation as the interface contract. Generated text is parsed for a Verilog code block; provided dependency modules that the model re-declared anyway are stripped before verification.

Internal verification mirrors the benchmark. The generator's inner loop lints each candidate with Verilator against the same file set the benchmark compiles — dependencies, reference model, and testbench, with the task Makefile's warning waivers applied — rather than linting the generated file in isolation. This design decision followed a concrete

doi:10.5566/NTU2026XXXXX

failure: an earlier isolated lint could not see testbench-side pin connections, so interface violations (`PINNOTFOUND`) sailed through internal checks and died only at evaluation. Mirroring the benchmark's compile closes that gap; on the runs where both verdicts exist for identical code, the internal lint agrees with the benchmark's syntax verdict on 60 of 60 tasks. The cost of this fidelity is that Verilator diagnostics can quote testbench or reference lines into repair prompts; we return to this leak surface in Section 7.2.

3.4 Repair Mechanisms

Three mechanisms sit on top of the generator's inner loop. Throughout the thesis, a repair budget "s/f" means at most s syntax-repair and f functional-repair attempts per sample.

3.4.1 Syntax Repair Loop and the Semantic Repair Cache

When the internal lint fails, the diagnostics are fed back to the model in a repair prompt together with the current candidate, and the process repeats up to the syntax budget. The loop is enhanced by a semantic repair cache (Figure 3.2) that reuses the model's own past successes: whenever a repair subsequently passes verification, the pair (diagnostic signature $\rightarrow$ unified-diff excerpt of the fix) is stored. Signatures are Verilator diagnostic lines with file paths and identifiers stripped, gated by error code, so that structurally identical errors on different modules collide. On a later failure, sufficiently similar cached entries are injected into the repair prompt — advisory only, never auto-applied, and never sourced from benchmark reference code. The cache has three scopes: `off`, `task` (shared only across one task's samples), and `run` (shared across the whole run). Run

doi:10.5566/NTU2026XXXXX

scope maximizes hint availability but couples a task's samples, which corrupts pass@k's independence assumption — the trade-off is measured in Section 6.4.

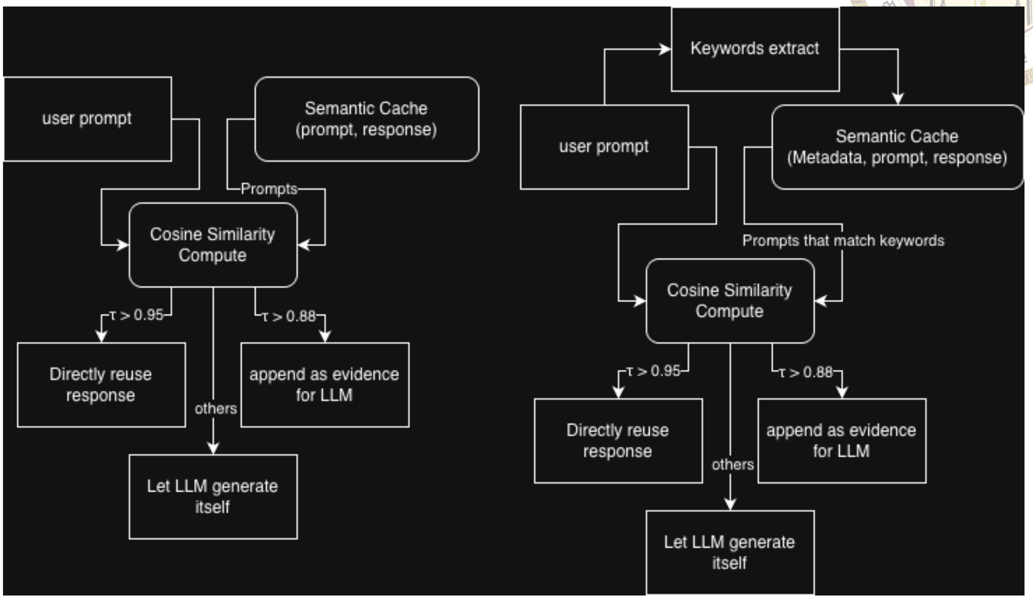

Figure 3.2: The semantic repair cache. Verified diagnostic→fix pairs from the model's own repairs are keyed by identifier-stripped diagnostic signatures and replayed as advisory hints when similar failures recur.

3.4.2 Functional Repair Loop

The syntax loop is structurally blind to the largest failure class: candidates that compile but fail the testbench (Section 6.8). The functional repair loop attacks this class directly. After a candidate passes syntax, the testbench is run; on mismatch, the simulator's mismatch report ("Output x has N mismatches, first at time T") is fed back in a prompt that frames the problem explicitly as logic repair — the interface is correct, do not change it — and the loop keeps the best compiling candidate by lowest mismatch count, never advancing to a candidate that regresses syntax.

One methodological property makes this loop different in kind from syntax repair, and we flag it prominently: the feedback signal comes from the same testbench that scores the benchmark. A pass obtained through functional repair therefore means "specification

doi:10.5566/NTU2026XXXXX

plus oracle feedback," not "specification alone." For that reason the loop is off by default and reported as a separate experimental arm (Section 6.5); headline pipeline-vs-baseline comparisons never include it.

3.4.3 Diagnostic-Driven Specification Slicing

Repair prompts initially carried the full specification, which for large tasks buries the relevant sentence under tens of thousands of irrelevant tokens. Specification slicing exploits the fact that Verilator diagnostics and testbench mismatch reports name the offending identifiers exactly. The slicer extracts those identifiers, then assembles a focused slice: the verbatim interface excerpts (port tables, module headers) plus only the specification sections that mention an offending identifier, under a character budget. When a diagnostic names no identifier (e.g., a bare parse error), the slicer returns nothing and the full specification is kept, so repair is never starved of context. On the largest specification the slice is a $\sim$29$\times$ reduction ($\sim$704 k $\rightarrow$ 24 k characters).

3.5 Summary

The pipeline's design distills to three commitments. Ground, don't trust: every reuse decision is resolved against the real catalog because models plan from memory even when handed a retrieval tool. Verify against the real environment: the inner lint compiles exactly what the benchmark will compile, making the two fatal contracts visible during repair rather than at scoring. Reuse the model's own verified successes: repair hints come from diffs that already fixed a structurally identical failure. What these commitments buy — and what they conspicuously fail to buy — is quantified in Chapter 6.

doi:10.5566/NTU2026XXXXX

Chapter 4 Cascade Routing between Direct and Pipeline Flows

The pipeline of Chapter 3 is not uniformly better than simply asking the model for the module. On a substantial class of tasks it is worse: its planning and reuse scaffolding invents structure that a self-contained module does not need, and derails a generation that a bare prompt would have gotten right. This chapter presents the second contribution of the thesis: a per-task cascade router that predicts, before spending pipeline-scale compute, which of two flows a task should take — the full agentic pipeline, or a cheap direct single-shot generation. Section 4.1 establishes why the two flows are complementary; Section 4.2 defines the task classes and the oracle labels used to evaluate routing; Section 4.3 describes the two-tier cascade; and Section 4.4 defines the routing-specific metrics used in Section 6.7.

4.1 Motivation: Two Complementary Flows

Running both flows on all 60 tasks (the Hybrid arm of Table 5.2) reveals a clean division of labor. With ten samples per flow, the pipeline alone solves 20 tasks and the direct flow alone solves 22 — but their union solves 30, because the two sets overlap only

doi:10.5566/NTU2026XXXXX

partially. Wrapper and integration modules pass under the pipeline, which supplies the exact interfaces of the submodules they must instantiate and wire; self-contained algorithmic modules pass under direct generation, which lets the model write the datapath in one coherent piece without the pipeline's decomposition machinery second-guessing it. Neither flow dominates: each one's strength is the other's failure mode.

The obvious exploitation of complementarity — run both flows on every task — is also the most expensive one, and it wastes most of its budget: every direct sample spent on a wrapper module and every pipeline run spent on a self-contained ALU is compute that predictably converts to nothing. A router that sends each task to its matching flow can, in principle, reach the union coverage at roughly half the cost. The routing problem is to make that prediction before the expensive flow has been paid for.

4.2 Task Classes and Oracle Labels

We formalize the division as two task classes:

Class A (structural). Wrapper, integration, and memory-encapsulation modules: the module's own logic is thin; its difficulty is wiring externally defined submodule interfaces correctly. Class A tasks want the **pipeline**.

Class B (algorithmic). Self-contained datapath or control modules — cipher rounds, CRC, instruction decode, address generation: the difficulty is the logic itself, not integration. Class B tasks want **direct** generation.

For evaluation only, each task receives an oracle label computed from its golden implementation: the reference RTL is synthesized with Yosys, submodule instances are

doi:10.5566/NTU2026XXXXX

subtracted, and the task is labeled A if its own synthesized logic falls below a 250-cell threshold (or it is a recognized regular-array wrapper), otherwise B. The label also fixes the task's target flow (A→pipeline, B→direct). Oracle labels use golden RTL that no generation-time component may see; they exist so that routing accuracy can be scored, and they define the ceiling arm (`oracle`) in the ablation of Section 6.7.

4.3 The Two-Tier Cascade

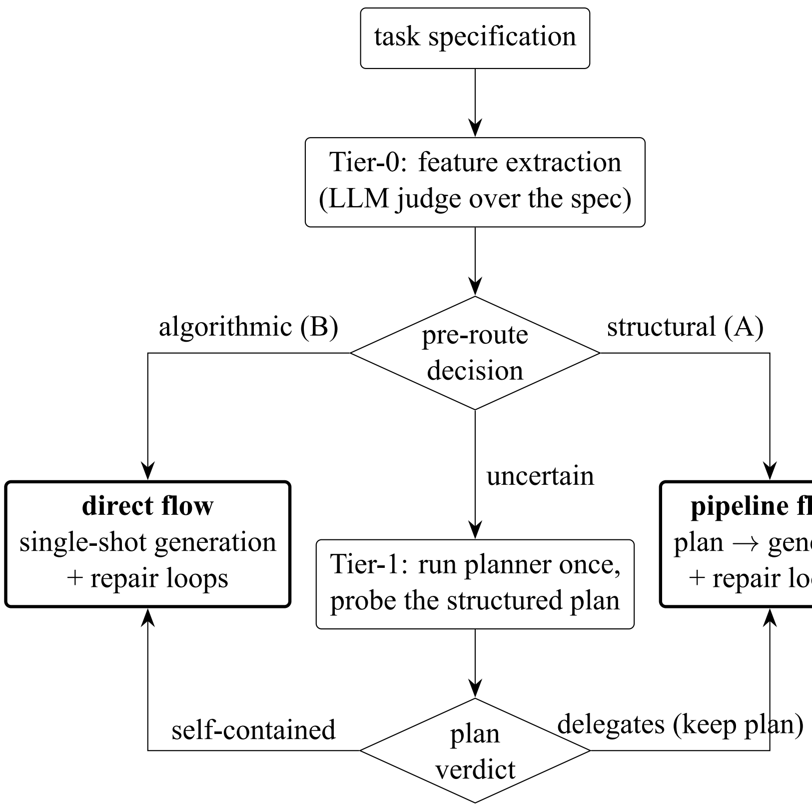

Figure 4.1: The two-tier cascade router. Tier-0 routes confident tasks from spec-level features alone; only uncertain tasks pay for a planner call, whose structured plan Tier-1 probes. A task routed to direct after planning discards the plan (a "wasted plan").

The router is a cascade of two increasingly expensive tests (Figure 4.1); a task exits at the first tier that is confident.

doi:10.5566/NTU2026XXXXX

Tier-0: spec pre-route. A feature extractor reads the specification and emits seven structured judgments: whether the module delegates to submodules, is a memory encapsulation, or is thin pass-through control (all class-A signals); whether it contains a finite-state machine, clock-domain crossing, or a datapath algorithm (class-B signals); and an estimate of the state bits the module itself must hold, plus a confidence value. The production extractor is a small LLM call that returns this JSON verdict (a regex-based keyword extractor over the same features serves as the ablation baseline — Section 6.7); the decision rule is deliberately transparent: any algorithmic signal routes to direct, else any structural signal routes to pipeline, else — or when the extractor's confidence is below threshold — the task falls through as uncertain. One subtlety the LLM extractor handles and keywords cannot: a module can sound algorithmic ("preliminary decoding") while actually being a wrapper that instantiates the real decoder, and only whole-spec reading resolves that.

Tier-1: plan probe. Uncertain tasks run the planner once — the cheapest informative pipeline component — and the router probes the resulting structured plan's own natural-language requirement summary. Decisive delegation phrasing ("wraps", "encapsulates", "reuses existing", or instantiation language without any own-implementation verb) keeps the task in the pipeline, plan in hand; clear self-contained-algorithm phrasing routes it to direct, discarding the plan. The asymmetry of the default is intentional: the plan is already a sunk cost, so the probe bails to direct only when the module is clearly self-contained. A plan generated and then discarded is counted as a wasted plan — the cascade's overhead metric.

The direct flow. The direct arm is not a bare completion: it shares the generator's repair machinery. A directly generated candidate that fails the internal lint enters the same

doi:10.5566/NTU2026XXXXX

syntax repair loop, and (in arms where functional repair is enabled) the same functional repair loop, with the plan-dependent prompt sections simply absent. This matters for the routing economics: direct generation costs roughly 10–12 k tokens per sample against the pipeline's 90–153 k, so every correctly-routed class-B task buys nearly an order of magnitude of compute reduction at equal or better quality.

4.4 Routing Metrics

Section 6.7 evaluates routing arms on four axes: coverage (solved tasks and pass@k, against the HYBRID union as the attainable ceiling); routing accuracy (the confusion between oracle class and chosen flow, where both misroute directions — A→direct and B→pipeline — forfeit a would-be pass); overhead (wasted plans); and efficiency (estimated tokens and summed compute per solved task). The design goal is stated once and measured throughout: reach the union coverage of both flows at the lowest compute, with misroutes near zero.

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Chapter 5 Experimental Setup

This chapter fixes the experimental ground rules shared by every result in Chapter 6: the benchmark and its scoring semantics (Section 5.1), the models and serving infrastructure (Section 5.2), the metrics and the cost-accounting rules we commit to (Section 5.3), and a complete inventory of the experimental runs together with their configurations (Section 5.4). The inventory table doubles as the reproducibility anchor of this thesis: every number reported later traces back to exactly one row of Table 5.2.

5.1 The RealBench Benchmark

All end-to-end evaluations use RealBench [9], a Verilog generation benchmark built from three real, open-source IP projects rather than from isolated textbook exercises. Each task provides a natural-language specification of one target module, the source files of the modules it may depend on, a golden reference implementation, and a self-checking testbench. The generator must produce the target module; the surrounding project files are supplied by the evaluation environment.

The benchmark distinguishes module-level tasks, which target one module inside an otherwise-provided project, from system-level tasks, which ask for an entire subsystem. This thesis reports on the **60 module-level tasks** summarized in Table 5.1; system-level

doi:10.5566/NTU2026XXXXX

Table 5.1: The 60 module-level RealBench tasks used in this thesis, by source project.

Family	Source project	Tasks	Character
AES	OpenCores AES-128 cipher core	6	self-contained datapath logic
SDC	OpenCores SD-card controller	14	bus-facing control logic
E203	Nuclei HummingBirdv2 E203 RISC-V core	40	deep module hierarchy, reuse-rich

tasks are excluded from all headline numbers because their specifications (up to $\sim$704 k characters for `e203_cpu_top`) exceed what any of our serving configurations can process without aggressive condensation, and because a 60-task universe with three distinct families is already large enough to expose the phenomena this thesis studies. The three families span a useful difficulty gradient: AES modules are small and algorithmically self-contained, SDC modules implement Wishbone-facing control logic with wide interfaces, and the E203 family covers a complete RISC-V processor core whose modules instantiate one another — the setting in which IP reuse has the most to offer.

Scoring. RealBench scores with its own per-task `Makefile`, and its verdict is deliberately strict. A sample is counted as a syntax pass only if Verilator [27] compiles the full project — generated module, provided dependencies, reference model, and testbench together — with **no `%Error` and no `%Warning`** outside the Makefile's explicit waiver list; warnings are fatal. A sample is a functional pass ("pass" for short) only if it also compiles and the testbench simulation reports zero output mismatches against the golden reference. Two consequences of this design recur throughout the thesis. First, our syntax numbers are conservative relative to benchmarks that fail only on errors (Section 7.2). Second, the environment imposes two non-negotiable structural contracts on generated code: a dependency that the environment ships as a file must be instantiated, never re-declared (re-declaration is a fatal `MODDUP`), and the testbench's DUT instantiation fixes the exact port list (a missing port is a fatal `PINNOTFOUND`). Section 3.3 describes how the pipeline

doi:10.5566/NTU2026XXXXX

internalizes both contracts.

5.2 Models and Serving Infrastructure

All generation uses open-weight models served locally through vLLM [11] behind an OpenAI-compatible endpoint, on university/NCHC GPU nodes reached through SSH port forwarding. Three models appear in this thesis:

- **gpt-oss-20B** [22] — the workhorse model. All ablations (grounding, repair cache, repair budgets, routing) run on this model, so their comparisons are model-controlled.
- **gpt-oss-120B** [22] — used to test which findings survive a $6\times$ scale increase (Sections 6.8 and 6.7).
- **Qwen3-235B** [33] — used in supplementary repair-budget runs; it does not participate in headline comparisons.

Two serving quirks shaped the implementation and are documented here because they are invisible in the results yet load-bearing for reproduction. First, the deployment returns answers in the `reasoning_content` field with an empty `content` field whenever tool schemas are attached to a request; the client transparently falls back to the reasoning field and, for the planner, re-prompts without tools (Section 3.2). Second, the server caps completion length rather than rejecting a request when prompt and requested tokens together exceed the 131,072-token context window, but returns HTTP 400 when the prompt alone exceeds it — the failure mode behind the "not generated" bucket of Section 6.8.

doi:10.5566/NTU2026XXXXX

5.3 Metrics and Cost Accounting

Quality. For a run with n samples per task, we report the syntax rate and pass rate as per-sample means over all generated records, and pass@k per task with the standard unbiased estimator [5]

$$\text{pass@}k \;=\; \mathbb{E}_{\text{tasks}}\left[1 - \binom{n-c}{k}\Big/\binom{n}{k}\right],$$

where c is the number of passing samples for the task, averaged over the 60 tasks. We additionally report solved tasks: the number of tasks with at least one passing sample, which equals $60 \cdot \text{pass@}n$ and is the quantity a practitioner who can afford n attempts actually cares about. Pass@k carries an independence assumption across a task's samples; one of our own techniques (the run-scoped repair cache, Section 6.4) violates it by design, so all reported pass@k values come from arms where samples are independent, and run-scoped numbers are shown only to quantify that specific effect.

Cost. We use two cost measures. Estimated tokens are computed from prompt/response character counts with a calibrated 4.38-characters-per-token heuristic and summed per run; compute time (Σwall) is the sum of per-sample wall-clock seconds, which measures total machine work independently of the harness's $\sim$80-way request concurrency. Three accounting rules apply throughout, each forced by a measurement hazard we identified in our own data: (i) token totals from runs that were resumed after interruption are not cited as costs, because resumed samples are re-read from disk with zero logged tokens; (ii) token totals for the direct flow are lower bounds, because its repair-loop calls are not instrumented; and (iii) wall-clock is never compared across runs executed on different days, because the shared vLLM server's load varied — only within-run sums and per-

doi:10.5566/NTU2026XXXXX

solved-task ratios are reported. These hazards and their blast radius are discussed together in Section 7.2.

5.4 Run Inventory

Table 5.2 lists every experimental run this thesis reports, the configuration that distinguishes it, and the section that consumes it. Short labels (left column) are used throughout Chapter 6. All runs share the same task discovery, per-task catalog construction, and RealBench evaluation harness, so rows differ only in the flow and the flagged configuration. Unless stated otherwise, sampled runs use temperature 0.2 (the runner default) and the two-stage pipeline of Chapter 3; "$6/4$" style entries abbreviate the syntax/functional repair-attempt budgets of Section 3.4.

Two rows deserve advance warning because they differ from their comparison partners in more than one variable. FUNC-10/10 changed both the repair budgets ($2/4 \to 10/10$) and a scoring waiver (`TIMESCALEMOD` warnings suppressed) relative to FUNC-2/4; Section 6.6 disentangles the two. And HYBRID predates the direct-flow repair loop that the router runs include, besides using the larger $10/10$ budgets; Section 6.7 therefore treats ROUTER-T1 $\to$ ROUTER-2T as the only single-variable routing comparison and labels the comparison against HYBRID as directional. One benchmark-side event also splits the timeline: the RealBench AES S-box testbench was repaired upstream on June 25, 2026 after we reported the race documented in Section 6.9; runs before and after that date are never compared on the two affected tasks.

doi:10.5566/NTU2026XXXXX

Table 5.2: Inventory of all reported runs. Every number in Chapter 6 traces to one row. Directory names are given in Table 5.3.

Label	Model	$60 \times n$	Temp.	Distinguishing configuration
Baselines and headline pipeline				
Direct-T0	20B	$\times 1$	0	spec $\rightarrow$ model, no plan, no repair
Pipe-T0	20B	$\times 1$	0	grounded planner + generator, cache off
Repair-cache trio (identical except cache scope)				
Cache-off	20B	$\times 20$	0.2	repair cache off
Cache-task	20B	$\times 20$	0.2	repair cache, task scope
Cache-run	20B	$\times 20$	0.2	repair cache, run scope
Model scale				
Pipe-120B	120B	$\times 10$	0.2	as Cache-run, tool-calling probe
Functional repair and budgets				
Func-2/4	20B	$\times 10$	0.2	functional repair + spec slice, budgets $2/4$
Func-10/10	20B	$\times 10$	0.2	as above, budgets raised to $10/10$
Routing (Chapter 4 arms)				
Hybrid	20B	$\times(10+10)$	0.2	no router: both flows on every task
Router-T1	20B	$\times 10$	0.2	plan-probe tier only, budgets $6/4$
Router-2T	20B	$\times 10$	0.2	full cascade (Tier-0 LLM + probe), $6/4$
Router-2T-120B	120B	$\times 10$	0.2	as Router-2T

Table 5.3: Mapping from run labels to repository directories under `runs/`.

Label	Directory
Direct-T0	`realbench_direct_model`
Pipe-T0	`agentic_plan_legacy_realbench_plan_hallu_fix_t0`
Cache-off	`agentic_plan_legacy_realbench_oss20b_plan_hallu_no_rep_cache`
Cache-task	`agentic_plan_legacy_realbench_oss20b_plan_hallu_task_rep_cache`
Cache-run	`agentic_plan_legacy_realbench_oss20b_plan_hallu`
Pipe-120B	`agentic_plan_legacy_realbench_oss120b_plan_hallu_tool_call`
Func-2/4	`repair_spec_slice_oss20b_func_v2`
Func-10/10	`repair_spec_slice_oss20b_func_v2_time-err-off_10syn_10func`
Hybrid	`hybrid_oss20b_10syn_10func_rep_spec`
Router-T1	`T1_router_oss20B_sync6_func4`
Router-2T	`Full-2T_router_oss20B_sync6_func4`
Router-2T-120B	`Full-2T_router_oss120B_sync6_func4`

doi:10.5566/NTU2026XXXXX

Chapter 6 Results and Analysis

This chapter evaluates the pipeline (Sections 6.1 through 6.6), the router (Section 6.7), and then turns the instruments on the failures themselves (Sections 6.8 and 6.9). All runs are identified by the labels of Table 5.2; all rates were recomputed directly from each run's per-sample records rather than transcribed from run logs.

6.1 Does Planning and Reuse Beat the Model Alone?

The cleanest head-to-head is deterministic: one greedy sample per task, identical harness, PIPE-T0 versus DIRECT-T0 (Table 6.1, Figure 6.1).

Table 6.1: Grounded pipeline vs. pure model on the 60 module tasks, temperature 0, one sample per task.

Arm	Syntax	Pass
PIPE-T0 (grounded planner + generator)	**37/60 = 61.7 %**	**15/60 = 25.0 %**
DIRECT-T0 (pure model)	16/60 = 26.7 %	6/60 = 10.0 %

The pipeline delivers $2.3\times$ the syntax rate and $2.5\times$ the pass rate of the same model prompted directly. The family breakdown locates the entire effect. On AES — small, self-contained, nothing to reuse — the model alone nearly saturates syntax (83 %) and the pipeline merely does not hurt (pass unchanged at 33 %). On SDC the pipeline helps modestly (pass 21.4 % $\rightarrow$ 28.6 %). On E203, the family whose modules instantiate one

doi:10.5566/NTU2026XXXXX

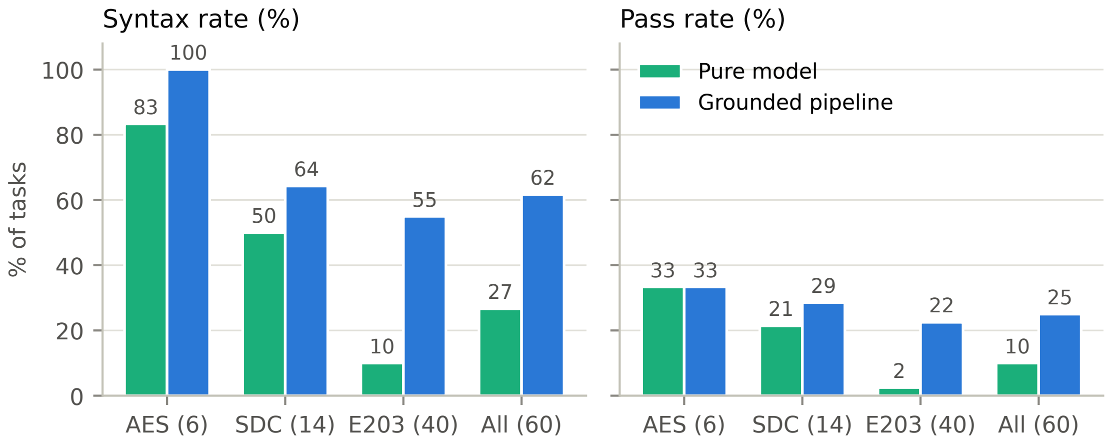

Figure 6.1: Syntax and pass rates by family, pure model vs. grounded pipeline (temperature 0, one sample). The aggregate gain is carried almost entirely by the reuse-rich E203 family.

another, the pipeline is decisive: syntax 10.0 % → 55.0 %, pass 2.5 % → 22.5 %. The conclusion we draw — and will lean on again when routing — is that the pipeline's value is proportional to how much legitimate IP reuse a task affords. Where there is nothing to ground, planning is overhead; where the task is an exercise in wiring real submodules, injecting their true interfaces roughly decuples the syntax rate.

6.2 The Grounding Audit, and an Honest Null Result

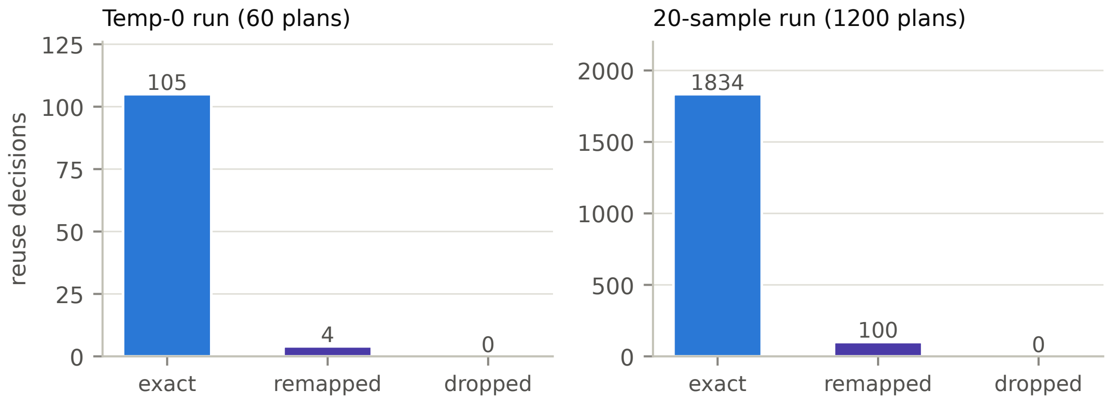

Figure 6.2: Grounding outcomes for every reuse decision in two runs. Near-miss names are remapped onto real catalog ids; nothing hallucinated survives into the plan.

Grounding-by-construction (Section 3.2) does exactly what it was built to do. In

doi:10.5566/NTU2026XXXXX

PIPE-T0, the 60 plans contain 109 reuse decisions: 105 already name an exact catalog id, 4 near misses are remapped, and 0 hallucinations survive (Figure 6.2). At twenty samples (CACHE-RUN), the same mechanisms process 1,934 decisions: 1,834 exact, 100 remapped, 0 leaked. Before the fix, plans routinely carried invented module names; after it, every reuse decision in every plan resolves to a real, provided module — and the planner's reuse matrix, previously empty because decisions hid under a dozen ad-hoc JSON keys, became fully populated and auditable.

And yet the end-to-end effect on quality is nearly zero: pass@1 moved from 19.6 % (ungrounded twin run) to 20.2 % — $+0.6$ percentage points, within noise. We report this null result deliberately. The generator was already silently discarding invented names and re-deriving reuse from the real dependency set, so grounding purchased plan correctness and auditability, not yield. Plan quality was not the binding constraint; Sections 6.8 and 6.9 show what is.

6.3 Sampling Structure: pass@1 to pass@20

Figure 6.3 shows the sampling structure of the pipeline on CACHE-OFF. Three facts matter downstream. First, diversity helps but saturates fast: pass@1 is 18.5 %, pass@5 already 26.6 %, pass@20 only 31.7 % — samples 6–20 add less than the first four. Second, the task universe splits sharply: 19 of 60 tasks are solved at least once in twenty tries; the other 41 never pass, at any k. Third, greedy decoding beats the sampled mean: the deterministic PIPE-T0 scores 25.0 % against the temperature-sampled per-sample mean of 18.5–20.2 % — the standard precision-versus-diversity trade (temperature is what buys the pass@20 headroom), with the caveat that the temp-0 and sampled runs also differ in

doi:10.5566/NTU2026XXXXX

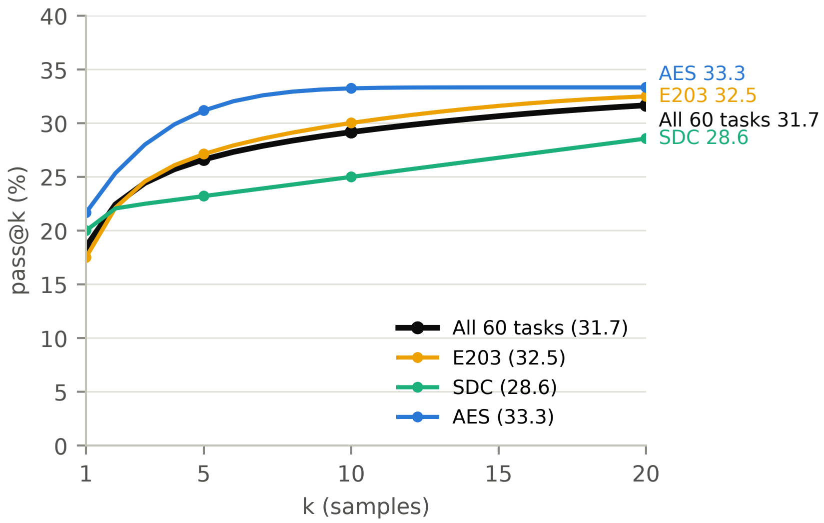

Figure 6.3: Unbiased pass@k on CACHE-OFF (60×20). Diversity buys $+13$ points from k=1 to k=20, but most of it is realized by k=5.

search mode, so the comparison is directional.

6.4 Repair-Cache Ablation: Density, Not Solvability

The cache trio (CACHE-OFF/-TASK/-RUN) differs in exactly one flag, giving the thesis's cleanest three-arm ablation (Table 6.2, Figure 6.4).

Table 6.2: Repair-cache scope ablation (60×20, gpt-oss-20B, identical configuration otherwise).

Cache scope	Syntax	pass@1	pass@20	mean repair attempts
off	59.4 %	18.5 %	**31.7 %**	1.12
task	64.3 %	19.6 %	**31.7 %**	1.32
run	66.9 %	20.2 %	**31.7 %**	1.61

The cache does what a syntax-hint mechanism can do: verified fix hints lift the syntax rate monotonically ($+4.9$ pp at task scope, $+7.5$ pp at run scope) and pull per-sample pass up with it, at the cost of more repair turns taken. What it cannot do is change which tasks are solvable: pass@20 is identical to the decimal across all three scopes. The cache shifts the density of passing samples inside already-solvable tasks; functional bugs — the actual

doi:10.5566/NTU2026XXXXX

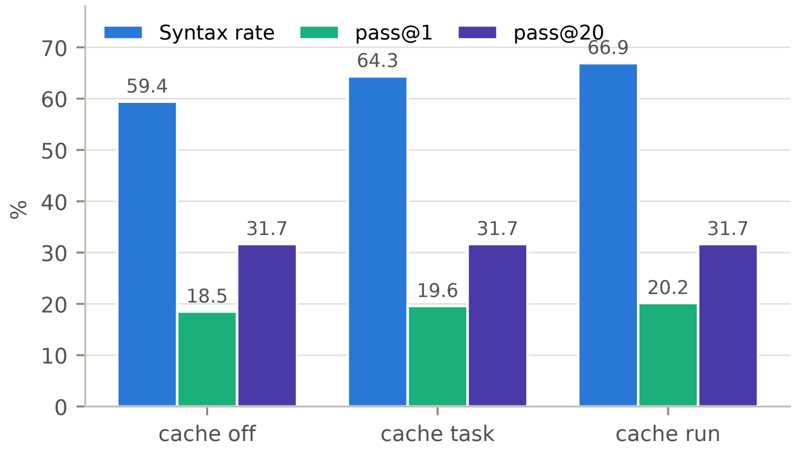

Figure 6.4: The cache lifts syntax monotonically and drags pass@1 with it, but pass@20 does not move at all.

frontier — are immune to syntax-diff hints.

Two qualifications. First, these are three independently sampled runs, so the monotone ordering of syntax and pass together is suggestive rather than proof; individual deltas of 1–2 pp are within sampling noise. Second, run scope shares hints across a task's twenty samples, violating pass@k's independence assumption — run-scope pass@k is shown only to demonstrate that even with that inflation pressure the solvable set does not budge. **Task scope is the methodologically safe default**, and it is what later runs adopt. Token totals across this trio are not comparable (the task/run arms were resumed mid-run; Section 7.2); the one clean cost figure is CACHE-OFF's 46.3 M estimated tokens for 1,200 samples.

6.5 Functional Repair: a Separate, Oracle-Fed Arm

The functional repair loop and specification slicing (Sections 3.4.2–3.4.3) are evaluated as their own arm because their feedback signal is the scoring testbench itself. Raising the budgets from $2/4$ (FUNC-2/4) to $10/10$ (FUNC-10/10) lifts syntax from 60.3 % to

doi:10.5566/NTU2026XXXXX

81.5 % and pass from 20.2 % to 27.5 % — but at 2.7× the cost (33.5 M → 89.8 M tokens). Attributing the +44 passing samples is sobering: +37 of them are candidates that, thanks to deeper syntax repair, now compile and then pass the testbench on their first functional check; only +7 net passes are attributable to the functional repair loop itself. (The two arms also differ in one scoring waiver — `TIMESCALEMOD` warnings suppressed in Func-10/10 — so the syntax delta slightly overstates the budget effect alone.) The loop's value proposition — oracle feedback included — is thin; the next section shows why.

6.6 Repair-Budget Calibration: Fix Fast or Never

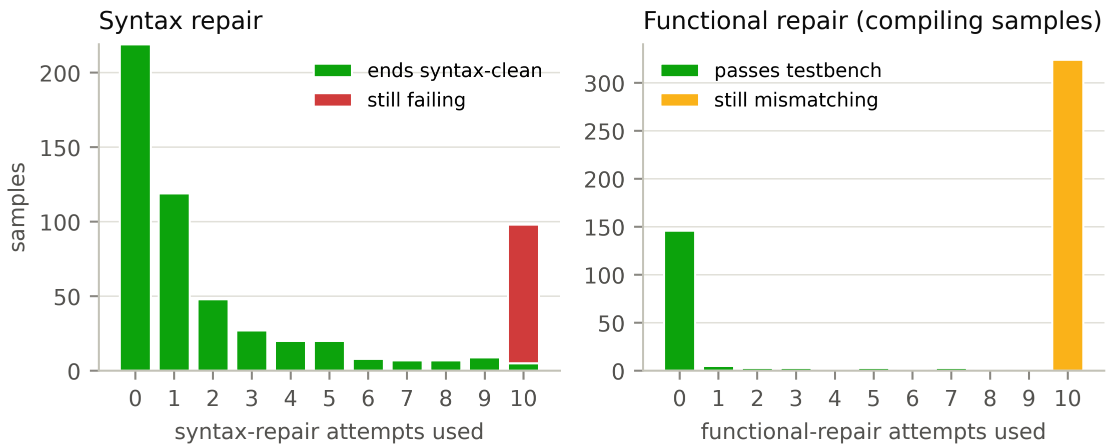

Figure 6.5: Func-10/10: outcomes by repair attempts consumed. Left: samples by syntax-repair attempts; nearly every bucket short of the cap ends syntax-clean, while the cap bucket (10) is 95 % failures. Right: compiling samples by functional-repair attempts; every rescue lands by attempt 7, and 324 samples burn the full budget converting nothing.

Both repair phases share one shape — fix fast or never — but with very different economics (Figure 6.5).

Syntax repair earns its budget. In Func-10/10, 219 samples compile without repair; repair rescues another 265 by attempt 9, with meaningful yield still arriving at attempts 4–6; the cap bucket is where hope goes to die (98 samples reach attempt 10; 5 succeed).

doi:10.5566/NTU2026XXXXX

The independent ROUTER-2T run confirms the shape at budget $6/4$: 202 born-clean, 234 repair-rescued with a real tail across all six attempts (77/54/44/23/22/14), 164 exhausted. A syntax budget of about six captures nearly all attainable value.

Functional repair does not. Of the compiling samples in FUNC-10/10 that enter the functional phase failing, the loop rescues 19 — every one by attempt 7 — while **324 samples burn all ten attempts and convert zero**. Since each attempt is a full compile-plus-simulation (up to 300 s) plus an LLM call, $3{,}240$ of $3{,}324$ functional-repair attempts — 98 % of the phase's compute — produced no pass. This is where the arm's $2.7\times$ cost blow-up lives. ROUTER-2T again confirms at scale: 13 functional rescues against 252 exhausted samples, and on its direct flow the loop is essentially dead (2 rescues in 342 samples). The prescription adopted for later runs: syntax cap ≈ 6, functional cap ≤ 4, plus a no-progress early exit (abort when the mismatch count stops dropping). The deeper reason functional repair fails so often — the mismatch signal itself is frequently uninformative or misleading — surfaces in Section 6.9.

6.7 Routing: Union Coverage at a Fraction of the Compute

Table 6.3: Routing arms (gpt-oss-20B). Solved = tasks with ≥ 1 passing sample. Σwall sums per-sample wall-clock. Token totals marked $^{+}$ are lower bounds (direct-flow repair calls are not instrumented). HYBRID and FUNC-10/10 use budgets $10/10$ and predate direct-flow repair; the single-variable comparison is ROUTER-T1$\rightarrow$ROUTER-2T.

Arm	Samples	pass@10	Solved/60	Σwall (h)	Mtok/solved
HYBRID (both flows)	600+600	46.3 %	30	423	3.24^{+}
FUNC-10/10 (pipeline only)	600	38.3 %	23	402	3.90
ROUTER-T1 (probe only)	600	35.0 %	21	213	2.17
ROUTER-2T (full cascade)	600	**56.7 %**	**34**	**130**	$\mathbf{0.83}^{+}$

doi:10.5566/NTU2026XXXXX

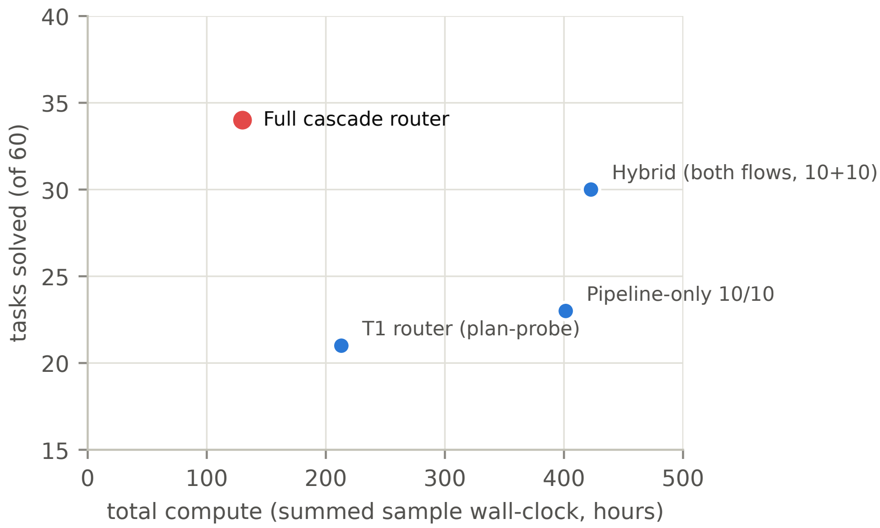

Figure 6.6: Tasks solved vs. total compute. The full cascade router solves the most tasks of any arm at less than a third of the compute of running both flows everywhere.

Table 6.3 and Figure 6.6 give the routing result. Three comparisons carry the argument.

Tier-1 alone is toothless. Router-T1 exposes only the plan probe, and the probe — which by design defaults to keeping the sunk-cost plan — routed every task to the pipeline. The arm therefore collapses to pipeline-only at budget $6/4$ (21 solved), and usefully so: it is the controlled base for the cascade.

The cascade is a strict win over its own ablation. Router-T1→Router-2T changes exactly one thing: Tier-0 LLM pre-routing in front of the probe (which then handles only the 5 of 60 tasks Tier-0 deferred). Solved tasks jump $21 \rightarrow 34$ and pass@10 $35.0\,\% \rightarrow 56.7\,\%$ while total compute falls $213 \rightarrow 130$ hours — more coverage for less work, because 342 of 600 samples run on the direct flow at roughly a tenth of a pipeline sample's token cost. Per solved task, the cascade spends $0.83\,\mathrm{M}$ tokens against the brute-force hybrid's $3.24\,\mathrm{M}$ — roughly $4\times$ more token-efficient.

doi:10.5566/NTU2026XXXXX

Against the hybrid ceiling, directionally ahead. ROUTER-2T solves 34 tasks where HYBRID — running both flows on everything — solves 30 at 3.3× the compute. This comparison is not single-variable: the hybrid predates the direct-flow repair loop (its direct samples compiled at 24.2 % vs. the router run's 76.0 %) and used $10/10$ budgets, so part of the router run's edge is repair, not routing. We therefore claim only that the cascade reaches at least union-level coverage at a fraction of the cost, not that it strictly beats the union. Notably, the two hybrid-solved tasks the router run misses (`e203_exu_branchslv`, `e203_ifu_ift2icb`) were correctly routed — both are direct-only tasks that the hybrid's pipeline flow also never solved (0/10 each) — and were lost to sampling variance on low-yield tasks (2/10 and 1/10 direct yields), not to the routing policy or the trimmed budgets.

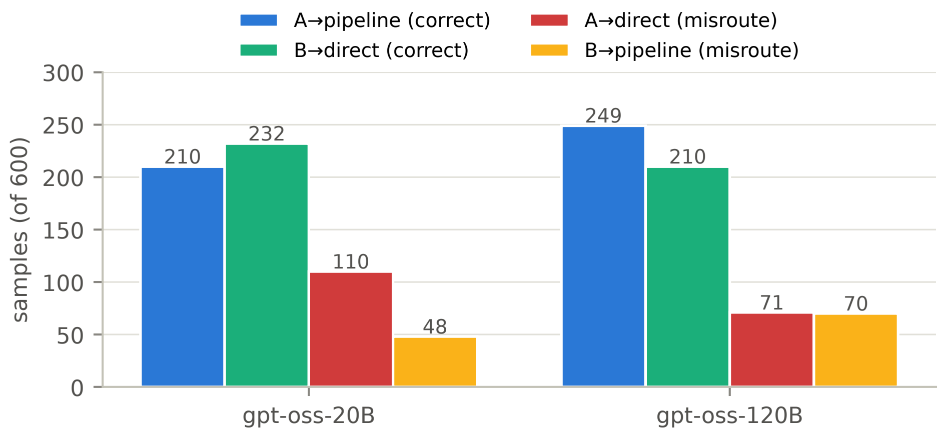

Figure 6.7: Flow allocation against oracle task classes for the full cascade, both model sizes. Misroutes concentrate in A→direct: pipeline-favoring tasks sent to the cheap flow.

Where the router is still wrong. Scored against the oracle labels (Figure 6.7), the 20B cascade misroutes 158 of 600 samples (26 %), dominated by A→direct (110): wrapper tasks sent to the flow that cannot see their submodule interfaces. The probe tier is underused (5 of 60 tasks), and 22 of 279 generated plans are discarded — the cascade's modest overhead. With gpt-oss-120B the same router solves the same 34 tasks at pass@1 41.2 % (vs. 30.7 %) and misroutes drop to 141, differently distributed (A→direct falls to

doi:10.5566/NTU2026XXXXX

71; B→pipeline rises to 70): the bigger model's feature extractor defers less and trusts the pipeline more. Closing the A→direct gap — e.g. by letting spec-level uncertainty trigger the probe more aggressively — is the router's clearest headroom.

6.8 Failure Taxonomy: Three Tiers of Difficulty

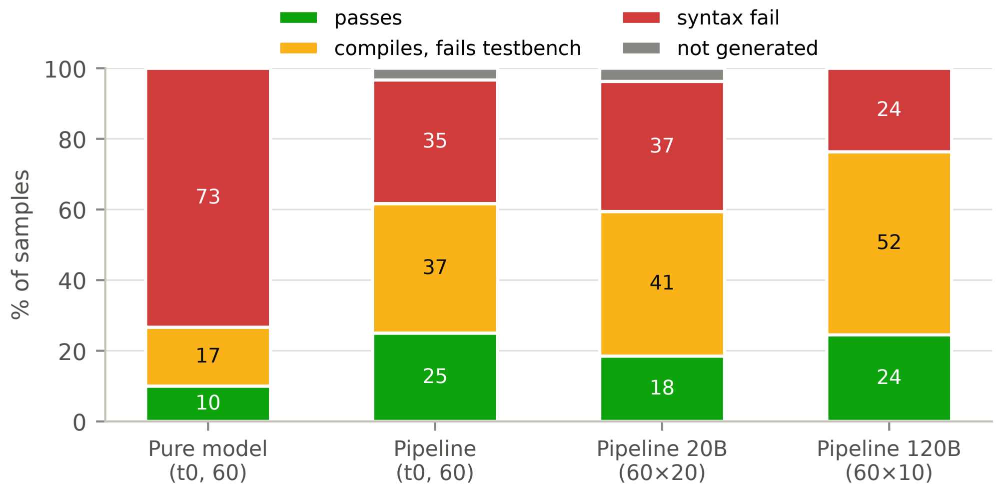

Figure 6.8: Per-sample outcomes. As models and repair improve, the red syntax band shrinks — and the amber compiles-but-fails-testbench band absorbs it. The functional share of compiling samples is essentially model-invariant.

Figure 6.8 decomposes every sample into four outcomes. The picture is consistent across runs: the largest failure bucket is not syntax but compiles-and-fails-the-testbench — right ports, right structure, wrong logic. Among compiling samples, that share is 59.5 % in PIPE-T0, 68.9 % in CACHE-OFF, and 67.9 % in PIPE-120B. The 6× larger model is a strictly better Verilog syntactician — syntax 59.4 % → 76.3 %, first-pass compiles up, repair turns down — but it is not a better logician: the functional share of compiling samples does not move. Scale fixes syntax, not semantics; this is the strongest evidence that the functional ceiling is structural rather than a capability artifact of the 20B model.

Reading failures per module (Figures 6.9 and 6.10) resolves the aggregate into three

doi:10.5566/NTU2026XXXXX

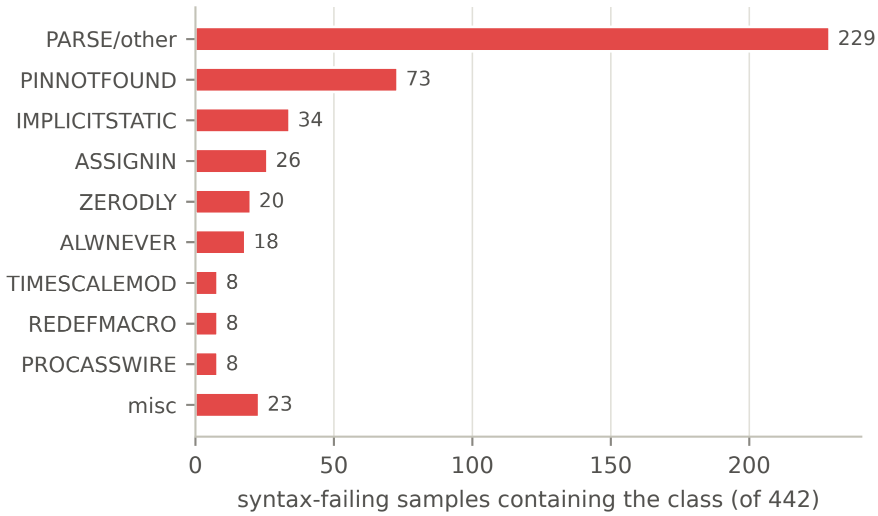

Figure 6.9: Verilator diagnostic classes among the 442 syntax-failing samples of CacheOff (a sample can carry several classes).

stable tiers:

1. **Solved: leaf, RAM, and glue modules.** `aes_key_expand_128`, the CRC and RAM blocks, clock gating, register files — small, well-specified, often pure reuse. The pipeline has essentially retired this tier.

2. **Logic-limited: datapath and control modules.** Most AES round logic and the E203 ALU/decode/commit/dispatch cluster compile with perfect syntax and fail the testbench — some at 0 passes despite dozens of structurally distinct compiling candidates. The syntax-only repair loop is blind here by construction.

3. **Scale-limited: giant integrators.** `e203_core/cpu/exu/lsu`, `sdc_controller`: hundreds of ports, dozens of submodules. Failures are interface-completeness (`PINNOTFOUND` concentrates exactly here), context overflow (the not-generated bucket: timeouts, empty returns, and HTTP 400s on the biggest specs), and integration logic. Notably, plan-level interface hallucination reappears only on this tier — the `e203_exu` plan promotes 25 internal signals to phantom top ports while dropping 10 real ones —

doi:10.5566/NTU2026XXXXX

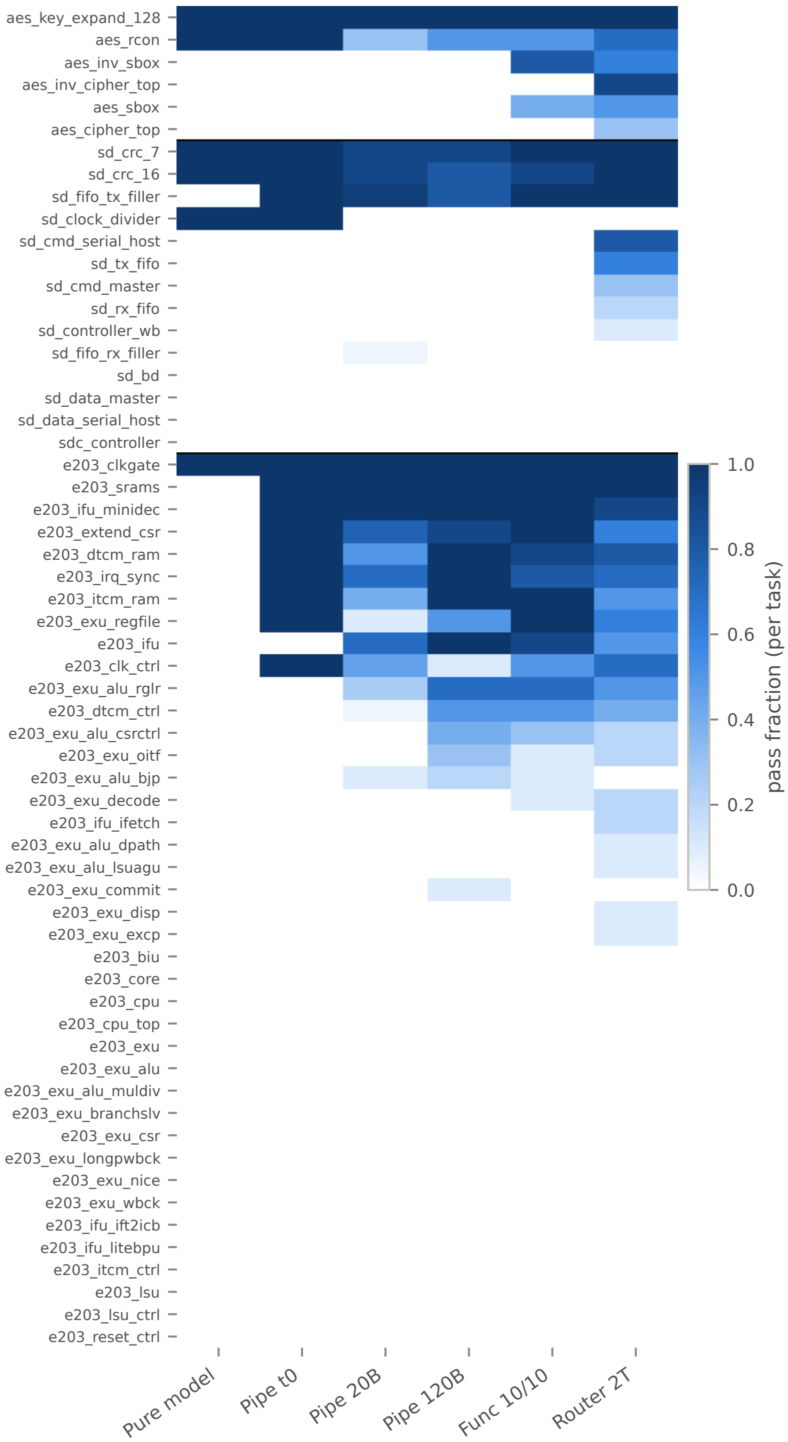

Figure 6.10: Pass fraction per task (rows, grouped AES/SDC/E203) across six runs (columns). The three difficulty tiers are visible as texture: saturated rows (leaf/glue modules), intermittent rows (logic-limited datapath modules), and blank rows (giant integrators).

doi:10.5566/NTU2026XXXXX

the same interface-too-large-to-hold phenomenon at a different stage.

Each tier wants a different intervention — nothing (tier 1), functional feedback (tier 2), decomposition and contract enforcement (tier 3) — which is why aggregate pass rate alone is a blunt instrument for steering this system.

6.9 When the Benchmark Is Wrong: Scoring Artifacts

Treating every 0 % module as model failure would misdirect the next round of engineering, so the persistent zeros were audited individually with a golden-model methodology: run the benchmark's own reference implementation through the full scoring path, and if the golden fails (or a byte-equivalent candidate fails), the task's zeros are artifacts, not model errors. Three audits produced three different verdicts.

The AES S-box race (confirmed artifact, since fixed upstream). `aes_sbox` and `aes_inv_sbox` scored 0 across all runs with one suspicious signature: every sample — combinational, ROM-array, or registered variants alike — reported exactly 766 mismatches. Diffing showed generated lookup tables byte-identical to the golden. The cause is a testbench race: stimulus changes the input and samples the output in the same simulation timestep, so any correct-but-differently-scheduled implementation (e.g. a continuous `assign` where the reference uses blocking `always @(a)`) settles one delta cycle late at each of the $\sim$768 input transitions. The trigger was isolated by incremental flag bisection to Verilator's `--coverage-line` instrumentation — the identical design passes without it — and the recorded mismatch count decomposes exactly as $766 + 3 \times$ (wrong table bytes), meaning the artifact also corrupts mismatch counts as a repair signal: a one-byte-wrong design

doi:10.5566/NTU2026XXXXX

(0.4 % wrong) reports ∼50 % mismatches. A registered-output module using the same testbench template (`aes_key_expand_128`) passes 20/20, completing the case that this is a combinational-settling race. Impact: 21 byte-perfect samples scored zero in CACHE-OFF alone, and 2 of 60 tasks were unsolvable purely by artifact. The finding was reported and the benchmark's testbench was repaired upstream on June 25, 2026 (settle before compare); pre-/post-fix runs are never compared on these two tasks.

The SDC controller (audited, genuine). `sdc_controller` — 0 functional passes in 220 records — looked like a candidate for the same treatment and is the reason the golden methodology matters: its golden implementation passes its testbench perfectly (0 mismatches in 23,932 checks) under the full scoring flags, because all its outputs are registered and immune to the settling race. The zeros are honest: 87 % of SDC samples die on syntax through interface hallucination introduced at the generation stage (instantiating provided submodules with invented pins, inventing a different master-bus naming, dropping the `sd_defines` macros) even when the plan's interface is correct — and the 13 % that compile are grossly wrong (∼95 % mismatch rates), not near misses.

The repair gate that fights the scorer (pipeline artifact). Auditing the 97 samples of FUNC-10/10 that exhausted all ten syntax repairs found that 42 of them (43 %) fail on a fatal warning located in a benchmark harness file (`stimulus_gen/ref`) — code the model cannot edit. The internal repair gate (`verilator --lint-only`) treats three warning classes as fatal that the actual scorer's build does not even emit, so on four tasks every sample burned its entire repair budget "fixing" correct code and never reached the functional phase. Golden test again: three of the four goldens pass the real scoring build (pure pipeline artifact; the fix is to mirror the scorer's waivers in the gate), while the fourth

doi:10.5566/NTU2026XXXXX

(`e203_reset_ctrl`) fails <u>even as golden</u> — a genuine benchmark defect of the AES class. The remaining 53 % of repair-exhausted samples fail on real errors in generated code, concentrated on the giant integrators of tier 3.

The methodological point generalizes beyond this benchmark: in a harness this strict, <u>persistent zeros deserve a golden-model audit before they are allowed to steer engineering effort</u> — of our three audits, one exonerated the models, one convicted them, and one convicted our own tooling.

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Chapter 7 Discussion, Limitations, and Conclusion

7.1 Discussion

What actually moved the needle. Ranked by measured effect, the levers were: (1) environment-faithful verification and repair — compiling against the real file set, injecting the testbench port contract, and condensing giant specs took the pipeline from low-single-digit to $\sim$18–25 % pass, the single largest step in the project's history; (2) routing — matching task character to flow shape lifted solved tasks from 21 to 34 while cutting compute by 39 % (Section 6.7); (3) model scale — 6$\times$ parameters bought +9 pp syntax and +4 pp pass@1 with the system held fixed; (4) repair caching — +5–7 pp syntax, density only. At the bottom: grounding (+0.6 pp, though it bought auditability and eliminated a failure class) and the functional repair loop (+7 net passes at 2.7$\times$ cost). The general lesson we draw is unglamorous: in agentic code generation for strict environments, fidelity between the inner loop and the scorer is worth more than additional intelligence in any single stage.

Grounding-by-construction over tool-calling. Every model we tested — from 20B to 120B — emitted structured plans directly and never once called the retrieval tool provided

doi:10.5566/NTU2026XXXXX

for exactly that purpose (0 of 1,800 plans). Designs that depend on the model choosing to invoke tools inherit that refusal as a silent failure mode: the plans looked plausible while citing modules that did not exist. The working alternative was to move the grounding out of the model's discretion — inject the vocabulary, constrain the output to it, and enforce it post-hoc. We suspect this pattern — make correctness a property of the harness, not a hoped-for behavior of the model — transfers well beyond RTL.

Why routing works here. The routing literature mostly buys quality with compute along one axis (small model vs. large model). Our setting is different in kind: the two flows have non-nested competence sets, so the expensive flow does not dominate and a router can beat both single-flow deployments at once — more coverage than either, at far less compute than running both. We expect this shape to recur wherever an elaborate agentic scaffold coexists with a cheap direct path: the scaffold helps structured, integration-heavy instances and actively hurts self-contained ones, so the routing signal (does this task's difficulty live in its interfaces or in its logic?) is legible from the specification itself.

The functional ceiling. The binding constraint at the end of this work is unambiguous: $\sim$60–69 % of everything that compiles fails its testbench, a share that neither model scale, nor caching, nor ten rounds of oracle-fed repair meaningfully dents. Verilator diagnostics give the repair loop a precise, local, actionable signal for syntax; a mismatch count gives almost nothing — and Section 6.9 showed it can even be actively misleading. Progress on this tier will require a fundamentally better feedback channel (waveform-level localization, per-submodule benches, assertion synthesis), not more attempts on the current one.

doi:10.5566/NTU2026XXXXX

7.2 Threats to Validity

We collect here, in one place, every caveat that qualifies the numbers in this thesis; each was also flagged inline where it bites.

1. **Single-seed runs.** Temperature-0 arms are one sample per task; each sampled run is one seed. Cross-run deltas under ∼2–3 pp are compatible with sampling noise. The claims we rest weight on (headline $2.5\times$; routing $+13$ tasks at -39 % compute; the flat pass@20) are far outside that band.

2. **Cross-run configuration drift.** Only three comparisons are single-variable: the cache trio, the temp-0 head-to-head, and ROUTER-T1→ROUTER-2T. Everything else — notably the router-vs-HYBRID comparison (repair budgets and direct-flow repair differ) and the FUNC-2/4 →FUNC-10/10 pair (a scoring waiver changed) — is labeled directional where it appears.

3. **Cost accounting.** Token totals from resumed runs undercount (resumed samples log zero tokens): the only clean full-run token cost is CACHE-OFF's 46.3 M. Direct-flow repair calls are not instrumented, so direct-flow token totals — including ROUTER-2T's 28.3 M — are lower bounds; the per-solved-task efficiency ratios in Table 6.3 would shrink somewhat, not reverse, under full instrumentation. Wall-clock is never compared across days (shared server, varying load).

4. **pass@k independence.** The run-scoped cache couples a task's samples; all load-bearing pass@k values come from cache-off or task-scoped arms.

5. **Scoring strictness.** The harness fails on warnings, so our syntax rates are conservative relative to error-only benchmarks — and Section 6.9 showed our own internal

doi:10.5566/NTU2026XXXXX

gate was at times stricter than the scorer, deflating four tasks.

6. **Benchmark fix boundary.** The AES S-box testbenches were repaired upstream on June 25, 2026, after our report. Every run in this thesis predates the fix except where stated; no comparison spans the boundary on `aes_sbox`/`aes_inv_sbox`.

7. **Oracle feedback.** The functional repair loop consumes the scoring testbench's mismatch report. Its passes mean "specification plus oracle feedback" and are quarantined in Section 6.5; no headline number includes them. Relatedly, the internal lint compiles the testbench, so Verilator diagnostics can quote testbench lines into repair prompts; a flag exists to disable this for leak-free claims, at a syntax-rate cost we did not pay here.

8. **Benchmark scope.** All results are on one benchmark's 60 module-level tasks and three IP projects, with open-weight models of one family for all controlled comparisons. The family-level heterogeneity we observe (AES vs. E203) is itself evidence that results will vary across codebases.

7.3 Future Work

Four directions follow directly from the measured constraints. First, functional feedback without oracle leakage: synthesize self-checking benches or assertions from the specification, so the functional repair loop gets a mismatch signal the scorer never sees — and gate it off pure-combinational modules, where Section 6.9 showed the signal can be corrupted. Second, interface-contract enforcement as a hard gate: the testbench DUT instantiation is already injected as text; promoting it to a mechanical post-generation port-diff check would delete the `PINNOTFOUND` class that dominates giant-module syntax failures,

doi:10.5566/NTU2026XXXXX

and the internal repair gate should mirror the scorer's warning waivers exactly. Third, selective decomposition for the giants: RealBench's system-level tasks decompose into submodules that are themselves benched module tasks, so a tree-structured generator could verify each node against a real testbench and compose upward — decomposition with per-node oracles, reserved for the scale-limited tier where it pays. Fourth, hardening the router: the dominant misroute (A→direct) argues for widening the uncertainty band that triggers the plan probe, and multi-seed replication would move the routing result from strongly suggestive to statistically settled.

7.4 Conclusion

This thesis asked how far careful system engineering — planning, grounding, verification, repair, and routing around fixed open-weight models — can push LLM-based RTL generation on a realistic, reuse-dominated benchmark. The answer is: substantially, and measurably. A two-stage pipeline that plans IP reuse against a grounded per-task catalog and generates under environment-faithful verification reaches $2.3\times$ the syntax rate and $2.5\times$ the pass rate of the same model prompted directly, with the entire gain concentrated where reuse exists to be exploited. A two-tier cascade router that matches task character to generation flow solves 34 of 60 tasks — more than running both flows on everything — at less than a third of the brute-force compute and $4\times$ better token efficiency per solved task.

The thesis's second kind of contribution is diagnostic honesty about the same system: grounding fixed the plans without moving the score; caching raised density without changing solvability; functional repair at depth ten wasted 98 % of its compute; and a

doi:10.5566/NTU2026XXXXX

golden-model audit of persistent zeros exonerated the models on two tasks, convicted our own repair gate on four, and confirmed the benchmark honest on the rest — findings that redirected the engineering each time, and one of which was adopted upstream by the benchmark itself. The frontier, at the end, is not syntax, not plans, and not scale: it is functional correctness, and the next system to move it will be the one that finds its models a better feedback signal than a mismatch count.

doi:10.5566/NTU2026XXXXX

References

[1] P. Aggarwal, A. Madaan, A. Anand, S. P. Potharaju, S. Mishra, P. Zhou, A. Gupta, D. Rajagopal, K. Kappaganthu, Y. Yang, S. Upadhyay, M. Faruqui, and Mausam. AutoMix: Automatically mixing language models. In Advances in Neural Information Processing Systems 37 (NeurIPS), 2024. arXiv:2310.12963.

[2] A. Allam and M. Shalan. RTL-Repo: A benchmark for evaluating LLMs on large-scale RTL design projects. In 2024 IEEE LLM Aided Design Workshop (LAD), 2024. arXiv:2405.17378.

[3] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-chat: Challenges and opportunities in conversational hardware design. In 2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD), 2023. arXiv:2305.13243.

[4] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. Transactions on Machine Learning Research, 2024. First appeared as arXiv:2305.05176 (2023).

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, et al. Evaluating large language models trained on code, 2021.

doi:10.5566/NTU2026XXXXX

[6] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, and A. H. Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In The Twelfth International Conference on Learning Representations (ICLR), 2024. arXiv:2404.14618.

[7] C.-T. Ho, H. Ren, and B. Khailany. VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 39, pages 300–307, 2025. arXiv:2408.08927.

[8] IEEE. IEEE standard for SystemVerilog—unified hardware design, specification, and verification language. IEEE Std 1800-2023, 2023.

[9] P. Jin, D. Huang, C. Li, S. Cheng, Y. Zhao, X. Zheng, J. Zhu, S. Xing, B. Dou, R. Zhang, Z. Du, Q. Guo, and X. Hu. RealBench: Benchmarking Verilog generation models with real-world IP designs, 2025.

[10] M. Keating and P. Bricaud. Reuse Methodology Manual for System-on-a-Chip Designs. Kluwer Academic Publishers, Boston, MA, 3rd edition, 2002.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2023.

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems 33 (NeurIPS), 2020. arXiv:2005.11401.

doi:10.5566/NTU2026XXXXX

[13] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu, et al. ChipNeMo: Domain-adapted LLMs for chip design, 2023.

[14] M. Liu, N. Pinckney, B. Khailany, and H. Ren. VerilogEval: Evaluating large language models for Verilog code generation. In 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD), 2023. arXiv:2309.07544.

[15] M. Liu, Y.-D. Tsai, W. Zhou, and H. Ren. CraftRTL: High-quality synthetic data generation for Verilog code models with correct-by-construction non-textual representations and targeted code repair. In The Thirteenth International Conference on Learning Representations (ICLR), 2025. arXiv:2409.12993.

[16] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie. RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. In 2024 IEEE LLM Aided Design Workshop (LAD), 2024. arXiv:2312.08617.

[17] S. Lu, N. Duan, H. Han, D. Guo, S. won Hwang, and A. Svyatkovskiy. ReACC: A retrieval-augmented code completion framework. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 6227–6240. Association for Computational Linguistics, 2022.

[18] Y. Lu, S. Liu, Q. Zhang, and Z. Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. In Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC), 2024.

[19] Nuclei System Technology. Hummingbirdv2 E203 ultra-low power RISC-V proces-

doi:10.5566/NTU2026XXXXX

sor core. https://github.com/riscv-mcu/e203_hbirdv2, 2026. Open-source project. Accessed July 2026.

[20] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is self-repair a silver bullet for code generation? In The Twelfth International Conference on Learning Representations (ICLR), 2024. arXiv:2306.09896.

[21] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. In The Thirteenth International Conference on Learning Representations (ICLR), 2025. arXiv:2406.18665.

[22] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025.

[23] OpenCores. OpenCores: Open-source IP cores repository. https://opencores.org, 2026. Accessed July 2026.

[24] H. Pearce, B. Tan, and R. Karri. DAVE: Deriving automatically Verilog from English. In Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD (MLCAD), pages 27–32, 2020.

[25] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany. Revisiting VerilogEval: A year of improvements in large-language models for hardware code generation. ACM Transactions on Design Automation of Electronic Systems, 30(6):91:1–91:20, 2025. arXiv:2408.11053.

[26] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems 36 (NeurIPS), 2023. arXiv:2303.11366.

doi:10.5566/NTU2026XXXXX

[27] W. Snyder. Verilator: Open-source SystemVerilog simulator and lint system. https://verilator.org, 2026. Accessed July 2026.

[28] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg. VeriGen: A large language model for Verilog code generation. ACM Transactions on Design Automation of Electronic Systems, 29(3):46:1–46:31, 2024.

[29] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri. AutoChip: Automating HDL generation using LLM feedback, 2023.

[30] Y.-D. Tsai, M. Liu, and H. Ren. RTLFixer: Automatically fixing RTL syntax errors with large language model. In Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC), 2024.

[31] C. Wolf and J. Glaser. Yosys – A free Verilog synthesis suite. In Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip), 2013.

[32] K. Xu, J. Sun, Y. Hu, X. Fang, W. Shan, X. Wang, and Z. Jiang. MEIC: Re-thinking RTL debug automation using LLMs. In Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2024. arXiv:2405.06840.

[33] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, et al. Qwen3 technical report, 2025. Qwen Team, Alibaba Group.

[34] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In Advances in Neural Information Processing Systems 37 (NeurIPS), 2024.

[35] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct:

doi:10.5566/NTU2026XXXXX

Synergizing reasoning and acting in language models. In The Eleventh International Conference on Learning Representations (ICLR), 2023. arXiv:2210.03629.

[36] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 2471–2484. Association for Computational Linguistics, 2023.

[37] Y. Zhao, D. Huang, C. Li, P. Jin, M. Song, Y. Xu, Z. Nan, M. Gao, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang, X. Zhang, Z. Du, Q. Guo, and X. Hu. CodeV: Empowering LLMs for Verilog generation through multi-level summarization, 2024.

doi:10.5566/NTU2026XXXXX

Appendix A — Prompt Templates

This appendix reproduces the load-bearing prompt text of the system, so that the mechanisms described in Chapters 3 and 4 can be audited at the level at which they actually operate. Text is excerpted verbatim from the repository (`agentic_ip_reuse/.../prompts.py` and `ip_reuse_legacy/prompts.py`); ellipses mark elisions.

A.1 Planner System Prompt (Catalog-Grounding Core)

The planner's system prompt walks the model through the eight planning stages and — decisive in practice — mandates the closed catalog vocabulary:

```
CATALOG GROUNDING (mandatory): The reusable-IP catalog for this
task is listed verbatim in the user message under "Reusable IP
catalog". Treat it as the only source of truth for which IPs
exist.
- Every reuse_decisions entry MUST set "selected_ip" to an
  "ip_id" that appears EXACTLY in that list.
- Never invent, rename, or add suffixes to an IP name (e.g. do
  not turn "e203_exu" into "e203_exu_core").
- If no catalog IP fits a module, set "new_rtl_required": true
  and "selected_ip": null for that module instead of guessing a
  name.
- Produce one reuse_decisions entry for every module that has a
```


doi:10.5566/NTU2026XXXXX

```
  candidate IP in the catalog.
...
Return final output as one JSON object only, with keys:
requirements, modules, reuse_decisions, integration_plan,
verification_plan, debug_plan, unresolved_assumptions.
```

Listing A.1: Planner system prompt, grounding section (excerpt).

The user message then carries the injected catalog digest (one line per provided module: `ip_id`, port summary, one-sentence description) and closes with: "Every reuse_decisions.selected_ip must be one of the ip_id values listed in the catalog above; do not invent names." As Section 6.2 showed, models comply with the vocabulary yet never call the retrieval tools also offered — which is why the post-hoc grounding pass of Section 3.2 remains in place as the enforcement backstop.

A.2 Generation Prompt Structure

The generator prompt is assembled from five blocks, in order: (1) the (possibly condensed) specification; (2) the source of every reused module, prefixed with the environment note that these modules are provided as files and must be instantiated, never re-declared; (3) environment notes (macros defined by the build, the one not-shipped dependency); (4) the testbench's DUT instantiation, verbatim, introduced as the binding port contract; and (5) the output-format instruction (one complete Verilog module in a fenced code block).

doi:10.5566/NTU2026XXXXX

A.3 Functional Repair Prompt

The functional repair turn (Section 3.4.2) re-frames the task as logic repair on a design whose interface is already correct. Its distinctive elements:

```
The following {target_hdl} design COMPILES and matches the
required port interface, but its outputs MISMATCH the reference
testbench:

<verbatim simulator mismatch report, e.g.
 "Output 'b' has 23 mismatches. First mismatch at time 115.">

Fix the LOGIC only. Do not change the module name or the port
list. Reused modules listed as provided source files must only
be instantiated, never re-declared in your output.

<behavioral slice: specification sections mentioning the
 mismatching output identifiers, plus verbatim interface
 excerpts (Section 3.4.3); full spec if no identifier named>
```

Listing A.2: Functional repair prompt, framing (paraphrased structure; mismatch report inserted verbatim).

When the repair loop runs on the direct flow (no plan), the reuse-framing block is dropped and the same skeleton applies — the mechanism by which the router's direct arm shares the pipeline's repair machinery (Section 4.3).

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Appendix B — Per-Module Results and Reproduction

B.1 Per-Module Pass Counts

Table B.1 lists, for every module task, the number of passing samples over generated samples in five representative runs (labels per Table 5.2). This is the tabular form of the heatmap in Figure 6.10; the table body is generated directly from each run's records by `scripts/thesis_figures/gen_module_table.py`.

Table B.1: Passing samples / generated samples per module task. "–" = task absent from the run.

Module	Direct-t0	Pipe-t0	Cache-off	Pipe-120b	Router-2t
`aes_cipher_top`	0/1	0/1	0/20	0/10	3/10
`aes_inv_cipher_top`	0/1	0/1	0/20	0/10	9/10
`aes_inv_sbox`	0/1	0/1	0/20	0/10	6/10
`aes_key_expand_128`	1/1	1/1	20/20	10/10	10/10
`aes_rcon`	1/1	1/1	6/20	5/10	7/10
`aes_sbox`	0/1	0/1	0/20	0/10	5/10
`sd_bd`	0/1	0/1	0/20	0/10	0/10
`sd_clock_divider`	1/1	1/1	0/20	0/10	0/10
`sd_cmd_master`	0/1	0/1	0/20	0/10	3/10
`sd_cmd_serial_host`	0/1	0/1	0/20	0/10	8/10

doi:10.5566/NTU2026XXXXX

Module	DIRECT-T0	PIPE-T0	CACHE-OFF	PIPE-120B	ROUTER-2T
sd_controller_wb	0/1	0/1	0/20	0/10	1/10
sd_crc_16	1/1	1/1	18/20	8/10	10/10
sd_crc_7	1/1	1/1	18/20	9/10	10/10
sd_data_master	0/1	0/1	0/20	0/10	0/10
sd_data_serial_host	0/1	0/1	0/20	0/10	0/10
sd_fifo_rx_filler	0/1	0/1	1/20	0/10	0/10
sd_fifo_tx_filler	0/1	1/1	19/20	8/10	10/10
sd_rx_fifo	0/1	0/1	0/20	0/10	2/10
sd_tx_fifo	0/1	0/1	0/20	0/10	6/10
sdc_controller	0/1	0/1	0/20	0/10	0/10
e203_biu	0/1	0/1	0/20	0/10	0/10
e203_clk_ctrl	0/1	1/1	9/20	1/10	7/10
e203_clkgate	1/1	1/1	20/20	10/10	10/10
e203_core	0/1	0/1	0/20	0/10	0/10
e203_cpu	0/1	0/1	0/20	0/10	0/10
e203_cpu_top	0/1	0/1	0/20	0/10	0/10
e203_dtcm_ctrl	0/1	0/1	1/20	5/10	4/10
e203_dtcm_ram	0/1	1/1	10/20	10/10	8/10
e203_extend_csr	0/1	1/1	15/20	9/10	6/10
e203_exu	0/1	0/1	0/20	0/10	0/10
e203_exu_alu	0/1	0/1	0/20	0/10	0/10
e203_exu_alu_bjp	0/1	0/1	2/20	2/10	0/10
e203_exu_alu_csrctrl	0/1	0/1	0/20	4/10	2/10
e203_exu_alu_dpath	0/1	0/1	0/20	0/10	1/10
e203_exu_alu_lsuagu	0/1	0/1	0/20	0/10	1/10
e203_exu_alu_muldiv	0/1	0/1	0/20	0/10	0/10
e203_exu_alu_rglr	0/1	0/1	5/20	7/10	5/10
e203_exu_branchslv	0/1	0/1	0/20	0/10	0/10
e203_exu_commit	0/1	0/1	0/20	1/10	0/10
e203_exu_csr	0/1	0/1	0/20	0/10	0/10

doi:10.5566/NTU2026XXXXX

Module	Direct-T0	Pipe-T0	Cache-off	Pipe-120B	Router-2T
e203_exu_decode	0/1	0/1	0/20	0/10	2/10
e203_exu_disp	0/1	0/1	0/20	0/10	1/10
e203_exu_excp	0/1	0/1	0/20	0/10	1/10
e203_exu_longpwbck	0/1	0/1	0/20	0/10	0/10
e203_exu_nice	0/1	0/1	0/20	0/10	0/10
e203_exu_oitf	0/1	0/1	0/20	3/10	2/10
e203_exu_regfile	0/1	1/1	2/20	5/10	6/10
e203_exu_wbck	0/1	0/1	0/20	0/10	0/10
e203_ifu	0/1	0/1	14/20	10/10	5/10
e203_ifu_ifetch	0/1	0/1	0/20	0/10	2/10
e203_ifu_ift2icb	0/1	0/1	0/20	0/10	0/10
e203_ifu_litebpu	0/1	0/1	0/20	0/10	0/10
e203_ifu_minidec	0/1	1/1	20/20	10/10	9/10
e203_irq_sync	0/1	1/1	14/20	10/10	7/10
e203_itcm_ctrl	0/1	0/1	0/20	0/10	0/10
e203_itcm_ram	0/1	1/1	8/20	10/10	5/10
e203_lsu	0/1	0/1	0/20	0/10	0/10
e203_lsu_ctrl	0/1	0/1	0/20	0/10	0/10
e203_reset_ctrl	0/1	0/1	0/20	0/10	0/10
e203_srams	0/1	1/1	20/20	10/10	10/10

B.2 Reproduction Commands

All runners live under `scripts/` in the project repository and require a live vLLM endpoint (`VLLM_BASE_URL`, `VLLM_MODEL`) plus Verilator on `PATH`. The commands below reproduce the thesis's principal arms; output directories match Table 5.3.

```
# Headline pipeline (temp 0) and pure-model baseline
```


doi:10.5566/NTU2026XXXXX

```
python scripts/run_agentic_plan_legacy_realbench.py \
  --task-level module --samples 1 --temperature 0 \
  --output-dir runs/agentic_plan_legacy_realbench_plan_hallu_fix_t0
python scripts/run_realbench_direct_model.py \
  --task-level module --samples 1 --temperature 0 \
  --output-dir runs/realbench_direct_model

# Repair-cache ablation (swap off | task | run)
python scripts/run_agentic_plan_legacy_realbench.py \
  --task-level module --samples 20 \
  --planner-search-mode hybrid --repair-cache task \
  --output-dir runs/..._task_rep_cache

# Grounding single-fix ablations
# --no-planner-inject-catalog | --no-planner-ground-reuse |
# --no-planner-completeness-gate

# Full cascade router (Tier-0 LLM decider + Tier-1 plan probe)
python scripts/run_realbench_routed.py \
  --router cascade --decider llm --samples 10 \
  --output-dir runs/Full-2T_router_oss20B_sync6_func4

# Per-module pass tables and figures
python scripts/module_pass_rate_report.py runs/<run_dir> \
  -o <run_dir>/module_pass_rate_analysis.md
cd scripts/thesis_figures && for f in fig_*.py; do python $f; done
```

Listing B.3: Reproducing the principal experimental arms.

doi:10.5566/NTU2026XXXXX